

National Technical University of Athens
School of Electrical and Computer Engineering
Division of Computer Science and Information Technology
Computing Systems Laboratory (CSLab)

SafeTree – A System for the Automatic Parallelization of Tree Data Structures

Diploma Thesis

by

Sotiris P. Dragonas

Supervisor: Georgios Goumas
Assistant Professor, NTUA

Athens, March 2020



National Technical University of Athens
School of Electrical and Computer Engineering

SafeTree – A System for the Automatic Parallelization of Tree Data Structures

Diploma Thesis

Sotiris P. Dragonas

Supervisor: Georgios Goumas, Assistant Professor, NTUA

Approved by the three-member examination committee on
3 April 2020.

Georgios Goumas – Assistant Professor
Nektarios Koziris – Professor
Dionysios Pnevmatikatos – Professor

Athens, March 2020.

Sotirios P. Dragonas
Diploma in Electrical and Computer Engineering, NTUA

Copyright © Sotiris P. Dragonas, 2020. All rights reserved. Copying, storing or distributing this work, in whole or in part, for commercial purposes is prohibited. Reproduction, storage and distribution for non-commercial educational or research purposes are permitted, provided that the source is acknowledged and this notice is retained. Questions concerning commercial use should be addressed to the author. The views and conclusions expressed in this document are those of the author and should not be interpreted as representing the official positions of the National Technical University of Athens.

Abstract

In this thesis, we present an automatic parallelization framework based on the RCU-HTM technique which allows the creation of thread-safe tree data structures while producing code very similar to their serial counterparts. The framework is designed as a C++ library and simplifies the design of parallel tree structures by hiding the complexity and hazards of parallel programming, while offering performance and scaling identical to the manual application of the RCU-HTM technique.

Keywords— RCU-HTM, parallel programming, concurrent data structures, hardware transactional memory, read-copy-update, automatic parallelization, C++ library

Acknowledgements

I would like to thank Dimitris Siakavaras for giving me the opportunity to undertake such an interesting diploma thesis, and for his invaluable help in completing it. I would also like to thank Professor G. Goumas for his support and for introducing me to the world of parallel programming.

Contents

1 Introduction 1
- 1.1 Definition of Concurrent Data Structures . . . 1
- 1.2 Motivation and Applications . . . 1
- 1.3 Challenges of Parallel Programming . . . 3
  - 1.3.1 Synchronization . . . 3
  - 1.3.2 Performance . . . 5
  - 1.3.3 Debugging . . . 7
- 1.4 Types of Concurrent Data Structures . . . 8
  - 1.4.1 Lock-Based Data Structures . . . 8
    - 1.4.1.1 Coarse-grained locking . . . 9
    - 1.4.1.2 Fine-grained locking . . . 11
  - 1.4.2 Lock-Free Data Structures . . . 11
- 1.5 Transactional Memory . . . 13

2 The Automatic Parallelization Library 15
- 2.1 Introduction and Goals . . . 15
- 2.2 The RCU-HTM Technique . . . 16
  - 2.2.1 Overview . . . 16
  - 2.2.2 The Algorithm . . . 17
- 2.3 Requirements for Users . . . 18
  - 2.3.1 Technical Requirements . . . 18
  - 2.3.2 Definition of Tree Structures . . . 18
  - 2.3.3 Tree Representation . . . 18
  - 2.3.4 Indexing a Node's Children . . . 19
  - 2.3.5 Required Node Methods . . . 19
    - 2.3.5.1 Minimum Requirements . . . 19
    - 2.3.5.2 Search-Tree Methods . . . 19
- 2.4 Basic Concepts . . . 20
  - 2.4.1 Operations . . . 20
  - 2.4.2 Read-Only Operations . . . 20
  - 2.4.3 The Original Tree . . . 20

2.4.4 Connection Point 20
2.4.5 The Copy Tree 20
2.4.6 Transaction 21
2.4.7 Validation 21
2.5 Core Library Components 23
2.5.1 Transaction Block 23
2.5.2 Connection-Point Search: Automatic and PathTracker 24
2.5.2.1 Search Trees 24
2.5.2.2 General Tree Structures 25
2.5.3 Connection-Point Search: Code Examples 26
2.5.3.1 Search in a General Tree 26
2.5.3.2 Search in a Search Tree 27
2.5.4 SafeNode (Safe Node) 28
2.5.4.1 Supported Methods 29
2.5.4.2 Using Safe Nodes 30
2.5.5 SafeNode: Code Examples 31
2.5.5.1 Example 1: Swap the Left and Right Children of a Binary-Tree Node 31
2.5.5.2 Example 2: Update a Value in a Key-Value Tree Node 31
2.5.5.3 Example 3: Swap Child Nodes if Their Keys Sum to an Even Number 32
2.5.6 ConnPoint (Connection Point) 33
2.5.6.1 Creating Safe Nodes 34
2.5.6.2 Moving Toward the Root 35
2.5.7 ConnPoint: Code Examples 37
2.5.7.1 Initializing a Connection Point 37
2.5.7.2 Reading and Installing a New Root 37
2.5.7.3 Tree Rebalancing with popPath 39
2.5.7.4 Code Example 40
2.6 Summary 41
2.7 Parallelizing an AVL-Tree-Based Map 42
2.7.1 Introduction 42
2.7.2 Required Includes 42
2.7.3 The Node Object: AVLNode 42
2.7.4 The Tree Object: AVLTree 44
2.7.5 Lookup 44
2.7.6 Insertion: Adding a Node 45
2.7.7 Insertion: Rebalancing a Sequential Tree 47
2.7.7.1 The Sequential Rebalancing Method 48

2.7.8 Insertion: Rebalancing Function for the Concurrent Tree . . . 49
2.7.9 Insertion: Rebalancing the Concurrent Tree . . . 53
2.8 Debugging . . . 55
2.8.1 Advantages of the Library . . . 55
2.8.2 Common Mistakes . . . 56

3 Technical Analysis of the Library 57
3.1 Overview and Required Methods . . . 57
3.2 Synchronization . . . 58
3.2.1 SpinLock . . . 58
3.2.2 TSXGuard . . . 59
3.2.2.1 Usage Example . . . 60
3.2.3 TSXTransOnlyGuard . . . 60
3.2.4 Transaction . . . 60
3.2.5 The Transaction Block . . . 61
3.3 Search . . . 62
3.3.1 ConnPointData . . . 62
3.3.2 Automatic Search Functions . . . 63
3.3.2.1 Lookup Functions . . . 65
3.3.3 PathTracker . . . 66
3.3.3.1 Initialization . . . 66
3.3.3.2 Structure . . . 66
3.3.3.3 Current Position and Connection Point . . . 66
3.3.3.4 Movement . . . 67
3.3.4 PathTracker Thread Safety . . . 69
3.4 The Copy Tree . . . 70
3.4.1 Developing the Idea . . . 70
3.4.1.1 Assumption . . . 70
3.4.1.2 Atomic Connection . . . 70
3.4.1.3 The Copy . . . 71
3.5 Safe Node . . . 71
3.5.1 Definition . . . 72
3.5.2 The getChild Method . . . 75
3.5.3 The setChild Method . . . 77
3.5.4 Relationship with ConnPoint . . . 78
3.5.5 Safe-Node Types . . . 78
3.6 Connection Point (ConnPoint) . . . 79
3.6.1 Connection-Point Structure . . . 79
3.6.2 Operations . . . 82
3.6.3 The Copy-Tree Root . . . 82

3.6.4 Connecting to the Original Tree . . . 83
3.6.5 Constructing Safe Nodes . . . 83
3.6.6 Validation . . . 84
3.6.7 Moving the Connection Point . . . 87
3.6.8 Completing an Operation . . . 90
3.7 Pools . . . 91
3.8 Early Operation Abortion . . . 93

4 Evaluation **94**
4.1 Code Size and Simplicity . . . 94
4.1.0.1 Manual RCU-HTM . . . 94
4.1.0.2 Library . . . 103
4.1.0.3 Assessment of the Concurrent Code . . . 107
4.2 Experimental Methodology . . . 108
4.3 Results . . . 108
4.3.1 Lookup Workloads . . . 109
4.3.1.1 Small Trees . . . 109
4.3.1.2 Medium Trees . . . 111
4.3.1.3 Large Trees . . . 113
4.3.2 Balanced Workloads . . . 115
4.3.2.1 Small Trees . . . 115
4.3.2.2 Medium Trees . . . 117
4.3.2.3 Large Trees . . . 119
4.3.3 Updates Only . . . 121
4.3.3.1 Small Trees . . . 121
4.3.3.2 Medium Trees . . . 121
4.3.3.3 Large Trees . . . 123
4.3.3.4 Performance Analysis . . . 124
4.4 Effect of NUMA on Performance . . . 124
4.5 Factors Affecting the Library's Performance . . . 124

5 Conclusions and Future Extensions **126**
5.1 Conclusions . . . 126
5.2 Future Extensions . . . 127

A Configuration **128**
Compilation Flags . . . 128
Numeric Parameters . . . 128
Operating Modes . . . 128
Optimizations . . . 128

Chapter 1

Introduction

1.1 Definition of Concurrent Data Structures

We define concurrent data structures as data structures that allow simultaneous access by multiple threads or processes without compromising their structure and while providing valid results.

1.2 Motivation and Applications

With the gradual slowing of Moore's law, modern designers of microprocessors and computer systems more broadly have turned to multicore systems, or even to the integration of heterogeneous systems to accelerate specialized tasks, in order to continue improving system performance. This work focuses on multicore systems.

For programmers developing software for such systems, however, parallel architectures introduce several challenges related to correctness, understanding parallel-programming techniques, and fully utilizing the available resources.

Focusing on data structures, a programmer quickly realizes that traditional data structures, such as a search tree, cannot be used unchanged in a parallel programming environment because of issues such as inter-thread synchronization, which we will discuss in the next section.

Data structures are unquestionably the foundation of most programs. Databases require structures for storing data on disk and others for their indexes; operating systems require priority queues for scheduling processes; and most algorithms for solving significant problems require

some form of data structure. For programs to operate on parallel architectures, they therefore need concurrent data structures.

1.3 Challenges of Parallel Programming

1.3.1 Synchronization

Synchronization refers to the requirement that certain parts of a program execute in a particular order, or in a particular order relative to one another. For example, in a program that calculates and prints the mean of an array's values, we want the result to be printed after it has been computed. In parallel code, however, that sequence cannot be taken for granted:

```
double average(double array[], int length, double* result) {
    *result = sum(array)/length;
}

int main() {
    double avg;
    double nums[100]

    /* initialize nums with numbers from 1 to 100 */
    thread avg_thread(average, nums, 100, avg);

    std::cout << avg << std::endl;
}
```

In the code fragment above, if execution were sequential, the programmer would expect the average to be printed. In a parallel setting, however, the printed value would most likely be some arbitrary value held by `avg`, because the original thread creates a new thread to compute the average but does not wait for the computation to finish before printing.

```
double average(double array[], int length, double* result) {
    *result = sum(array)/length;
}

int main() {
    double avg;
    double nums[100]

    * initialize nums with numbers from 1 to 100 */
    thread avg_thread(average, nums, 100, avg);

    avg_thread.join();

    std::cout << avg << std::endl;
}
```

With the addition of the new statement, the original thread waits for the computation to finish.

This deliberately simplistic scenario is one of the easiest cases in which synchronization is needed. In more complex situations, including the producer-consumer and dining-philosophers problems, as well as real production systems that may perform critical tasks such as financial transactions, implementing correct synchronization is a genuine challenge.

1.3.2 Performance

As mentioned earlier, a principal motivation for exploiting parallel systems is higher performance. Ideally, adding computational units, such as processor cores, would increase performance linearly with the number of units. Without careful programming, however, performance can actually fall below that of a single unit. Some of the factors influencing performance are:

1. **Amdahl's law**

   Amdahl's law states the following for parallel systems. If a task can be divided into work that may run in parallel and work that must run sequentially, and the parallelizable work accounts for a fraction p of the total sequential execution time, then the maximum speedup attainable using n parallel units is:

   $$\frac{1}{(1-p)+p/n}$$

2. **The memory system**
   One mistaken assumption in parallel programming is that the memory system does not matter and that all memory accesses behave as they do on a conventional sequential architecture. On the contrary, the memory system can significantly affect program performance. Examples include:

   - **The cache**
     For a thread to access a memory location modified by a thread running on another core, data must be transferred between cores to maintain cache coherence. Moreover, communication between cores takes place through an interconnect, which can be thought of, in simplified terms, as a communication bus with limited bandwidth and the ability to serve only a certain number of simultaneous requests. Inefficient accesses to shared memory can therefore produce real differences in performance.
   - **Non-uniform memory access (NUMA)**
     On some systems, such as multisocket machines, access to memory is not uniform across processing units. For example, when a core on one socket accesses data in the L3 cache of another socket, the data must cross an interconnect, incurring

much greater latency than direct access to the L3 cache on its own socket.

3. **Contention for resources**
   Contention plays an important role in the performance of a parallel system. Resources, whether input/output devices or in-memory data structures, can generally support only a limited number of simultaneous accesses. Increasing contention for these resources reduces performance. Particularly in the context of concurrent data structures and this thesis, maximum performance requires keeping contention to a minimum.

4. **Data organization**
   Data organization is an important factor in the performance of a parallel program. Ideally, data should be structured to minimize the sharing of memory among processors, reducing both contention and cache conflicts. This can be achieved through programming techniques, such as maintaining separate data for each processing thread and merging the results, or by changing the physical organization of data, such as aligning structures to prevent different cores from sharing the same cache line (false sharing).

5. **Synchronization mechanisms**
   Mechanisms used to achieve synchronization affect performance because they may introduce additional latency. As discussed in the next section, polling-based mechanisms waste processing cycles checking whether conditions have changed, whereas event-based mechanisms that suspend threads incur context-switch costs when those threads are awakened.

1.3.3 Debugging

Perhaps the greatest challenge in parallel programming is debugging. The complexity of interactions between threads often makes a program failure difficult to reproduce across multiple runs. Furthermore, the number of possible execution interleavings grows combinatorially as the number of threads rises, making a concurrent program difficult to understand. A characteristic example is the most common debugging technique, inserting print statements into the code: without synchronization, output from different threads becomes a jumble of interleaved character sequences.

1.4 Types of Concurrent Data Structures

1.4.1 Lock-Based Data Structures

The most widely used concurrent data structures are lock-based. Locking uses primitives such as mutexes and spinlocks to create mutually exclusive regions of code in which only one of the active threads may execute at a time.

Before showing examples of these data structures, we will briefly discuss these two primitives.

Mutex A mutex is a synchronization mechanism usually implemented with operating-system support. It provides two operations: locking and unlocking.

When a thread encounters a lock instruction, it atomically checks a memory location or operating-system structure indicating whether another thread is executing in the protected region. *If so*, the thread or process is suspended (for example, by being placed in an operating-system queue) until an unlock operation occurs. *Otherwise*, it proceeds, first recording that it has entered the region. This establishes mutual exclusion.

Mutexes are widely used for locking, and are generally avoided only where suspension is prohibited (for instance in an operating-system interrupt handler), or when the protected region is so short that its execution time is comparable to the overhead of the mutex itself. In those situations, spinlocks are used.

Spinlock A spinlock is a very similar synchronization mechanism, but it polls to determine when entry to the mutually exclusive region is permitted, so a waiting thread does not suspend execution. Spinlocks are usually implemented using atomic instructions.

Beyond these two varieties of lock, there are numerous variants aimed at challenges in parallel programming. These include contention-management locks, reader-writer locks that allow multiple concurrent reads but prohibit conflicting reads and writes or multiple writes, and many others. Corresponding concurrent data structures exist. For the purposes of this thesis, however, we classify lock-based structures into two broader categories.

1.4.1.1 Coarse-grained locking

Concurrent data structures using coarse-grained locking are the simplest application of the primitives described above. Starting from a sequential implementation, the entire code of each operation—insertion, lookup, and so on—is placed inside a mutually exclusive region. There is usually a single synchronization primitive per data-structure instance, permitting only one thread at a time to access it.

Other designs, such as reader-writer locks, may permit concurrent reads, but the defining characteristic of coarse-grained locking is that it does not exploit the internal organization of the data structure to enable more parallelism.

Its disadvantage follows directly: only one thread may modify the structure at a time, serializing access and preventing performance improvements when more threads are used.

Its advantages are simplicity of implementation and thread safety, which can be sufficient for structures where maintaining consistency matters more than access speed.

Implementation example Consider a `LinkedList` with methods `void insert_at_end(int new` `void delete_at(int index)` and `int index_of(int element)`. A concurrent version could be implemented in C++ as follows:

```
// ThreadSafeLinkedList inherits from LinkedList
class ThreadSafeLinkedList: LinkedList {
    private:
        Mutex mut;
    public:
        // initialize a linked list and a mutex as unlocked
        ThreadSafeLinkedList(): LinkedList(), mut(UNLOCKED){}

        void insert_at_end(int new_element) {
            // lock
            mut.lock();

            // call method of linked list
            LinkedList::insert_at_end(new_element);

            // unlock
            mut.unlock();
        }

        void delete_at(int index) {
            mut.lock();
            LinkedList::delete_at(index);
            mut.unlock();
        }

        int index_of(int element) {
            mut.lock();
            int index = LinkedList::index_of(element);
            mut.unlock();

            return index;
        }


}
```

Listing 1: Coarse-Grained Locking Linked List

1.4.1.2 Fine-grained locking

Fine-grained locking refers to concurrent data-structure implementations that **exploit their internal organization to enable parallel access through locks**. These designs often use multiple synchronization primitives according to the internal layout of the data. In trees, for example, each node may have its own lock, requiring locks to be **coordinated to guarantee thread-safe access**. Combining primitives while exploiting structural properties makes it possible to **achieve high degrees of parallelism** [CGR13],[Bro+10],[WSJ18].

This approach, however, brings considerable difficulties. Coarse-grained synchronization is simple enough that a novice programmer could implement it from a sequential data structure. Designing fine-grained concurrent structures, by contrast, tends to be the domain of exhausted PhD students and researchers.

Interactions between code inside and outside mutually exclusive regions, combined with **interactions between separate mutually exclusive regions**, create a rather chaotic situation. **Execution order becomes difficult to understand, deadlocks arise, and mistakes can easily corrupt the structure**; to add the icing to the parallel cake, **debugging is difficult because runs of the same code may yield radically different results**.

Further expertise is required to achieve better scalability than coarse-grained locking, since **the number of locks must be minimized**: each additional lock pushes performance back toward the coarse-grained case. There is also a **performance cost to locking**, both in processor cycles spent inside synchronization primitives and where those primitives do not schedule access optimally, for example because of ineffective contention management.

Naturally, the benefits include the potential for high parallelism and performance, the availability of lock libraries in most programming languages, and their maturity, since locks remain the most widely used method for building concurrent data structures.

1.4.2 Lock-Free Data Structures

As their name suggests, lock-free data structures are intended to avoid locks in their implementation, thereby avoiding blocking, or the waiting caused by locks.

They are usually built with atomic instructions and can provide valuable progress guarantees compared with lock-based structures. These

guarantees are commonly divided into three categories [Nam16]:

- **Obstruction freedom**: the algorithm guarantees that a thread will make progress on its operation as long as no conflicts arise; it will not have to wait for another thread's actions.
- **Lock freedom**: the operations of the structure make overall progress, meaning that at least one operation is guaranteed to complete in finitely many steps. This combines obstruction freedom with the absence of livelock.
- **Wait freedom**: progress is guaranteed for every operation individually; each completes in finitely many steps. It combines lock freedom with freedom from starvation.

At first glance, lock-free structures appear to be the ideal implementation of concurrent data structures. In practice, they introduce significant difficulties.

- **Implementation difficulty**. Lock-free algorithms are harder to understand and verify. In the absence of mutually exclusive regions, execution order cannot be controlled directly. Thread safety must be obtained through appropriate atomic checks during execution to preserve the structure's consistency. The greatest difficulty, however, is that atomic instructions cannot generally be composed into larger atomic operations.
- **Performance**. Synchronization algorithms for lock-free structures often retry operations multiple times until conflicts disappear, or are sufficiently complex that they perform worse than lock-based implementations.
- **Major differences from structure to structure**. Lock-free implementations often take quite different forms depending on the structure being implemented. As a result, implementation techniques do not readily transfer from one data structure to another. They consequently become the exclusive territory of exhausted PhD students.

Nevertheless, substantial research has been devoted to this field (see [CNT14], [RM15]). In particular, work on a general framework for building lock-free trees ([BER17]) inspired the data organization used by the library presented here.

1.5 Transactional Memory

Transactional memory uses transactions, rather than locks or individual atomic instructions, to achieve synchronization. Transactions are sets of memory accesses (reads and writes) performed atomically.

There are two approaches to implementing such transactions:

- **Hardware Transactional Memory (HTM)**, which uses mechanisms provided by the processor.
- **Software Transactional Memory**, which implements the necessary logic in software using other synchronization mechanisms as building blocks.

Since this thesis is based on HTM, only that method is discussed here.

How does synchronization work? First, a way is needed to designate a section of code to execute as a transaction. Intel's implementation provides special instructions in the processor ISA marking the beginning and end of a transaction and defining intervening memory accesses as transactional.

On first access, each region of memory (at cache-line granularity in Intel's implementation [Sia+15], since the mechanism extends cache coherence) is added to either a *read set* or a *write set*, according to the type of access. If another thread writes to a location belonging to either set while the transaction is executing, the transaction aborts. The affected memory locations revert to the state before the transaction, and execution may be retried. In this way, instructions within a successful transaction execute atomically.

The advantage is that critical sections which would otherwise require mutual exclusion may execute concurrently when their memory accesses do not conflict. On an ideal machine, it would therefore be possible to wrap all operations on a data structure in transactions and obtain fine-grained-locking performance with the simplicity of coarse-grained locking.

Unfortunately for performance—but fortunately for those of us studying these systems—real implementations have limitations that prevent such ideal behavior, requiring additional work to achieve efficient execution.

Some of these limitations are:

1. **Set capacity**. The read and write sets have a limited hardware capacity. A transaction that exceeds it must abort, so only a

limited number of memory accesses can occur within a transaction. These failures are called capacity conflicts.

2. **Progress**. Commercial implementations [Sia+15] offer only best-effort transactional completion; a transaction is not guaranteed to succeed however often it is retried. A fallback mechanism that guarantees completion is therefore necessary, such as taking a lock after a fixed number of failures.

3. **Reasons for transaction abortion**. Real transactions also abort for many reasons unrelated to conflicting data accesses, such as system interrupts, page faults, exceptions, context switches, and explicit user aborts. The synchronized code therefore affects whether it can execute transactionally at all, let alone efficiently.

Despite these limitations, careful use of the mechanism can create concurrent data structures with **high performance that scales linearly with the number of accessing threads**, provided they access independent regions of memory. Moreover, as we will demonstrate, the resulting code can closely resemble its sequential counterpart, a major advantage of the method.

Chapter 2

The Automatic Parallelization Library

2.1 Introduction and Goals

This thesis presents an automatic parallelization framework that greatly facilitates implementing concurrent data structures based on trees. Its concrete implementation is a C++ library available at https://github.com/BazookaMusic/RcuHTMParallelTrees.

Its design objectives are:

1. Handle the hazards of parallelism inside the library rather than exposing them to the user.
2. Keep the resulting code as close to its sequential counterpart as possible, in both readability and appearance.
3. Avoid any performance compromise compared with manual RCU-HTM implementations (a zero-cost abstraction).
4. Make debugging easier for the end user.

2.2 The RCU-HTM Technique

This thesis is based on RCU-HTM, a technique developed by my advisor Dimitris Siakavaras together with the Computing Systems Laboratory (CSLab). The following account draws on the official paper [Sia+17].

2.2.1 Overview

RCU-HTM combines Read-Copy-Update (RCU) [Des+12] with Hardware Transactional Memory (HTM) to **implement efficient concurrent search trees**.

This thesis **extends its applicability to arbitrary tree structures**, subject to certain restrictions. As with RCU-based algorithms, **tree modifications are made to copies in the threads' private memory** (see "copy trees") rather than to the original tree. This allows reader threads to traverse the tree without synchronization and without being affected by concurrent writers. At the same time, **HTM lets multiple writers connect their copies atomically**, enabling more than one writer thread.

This is the technique's main advantage over earlier RCU-based algorithms. **After the appropriate modifications have been made to the private copies, an atomic transaction verifies that every part of the tree affected by the change has remained unchanged since it was read. If validation succeeds, the copies are installed**.

2.2.2 The Algorithm

- **Traversal**. The first phase traverses the original tree outside a transaction to locate the node to which the private copy will later be attached. This node is called the connection point. For search operations, the algorithm ends here; for update operations, the path to the connection point is saved.

- **Copy construction**. Each thread constructs a local copy of the subtree to be modified, and performs the changes in that copy. At the same time it retains references to the original nodes underlying the copies, together with saved child-pointer values for validation.

- **Validation**. After constructing the copy, the thread begins an HTM transaction. Validation has two stages. First, it locates the connection point and verifies that it still belongs to the original tree, using the saved traversal path or properties of the tree (such as the node key). Second, it checks that the children of modified nodes have not changed, using the original nodes and the previously saved child-pointer values.

- **Connection**. If validation succeeds, the copy is attached atomically and the transaction completes. Otherwise, the entire process is restarted until it succeeds.

2.3 Requirements for Users

2.3.1 Technical Requirements

A compiler supporting at least C++11 is required. The processor must also support Intel's TSX-NI instruction set. Future versions may support other hardware transactional memory implementations.

2.3.2 Definition of Tree Structures

Here, tree structures are those that can be defined recursively as a root node with n children, which are themselves trees, with n bounded by some fixed maximum. Null child references are represented by `nullptr` pointers.

In addition, an implementation must provide an object that serves as a tree node and supports certain methods. We refer to this object as *Node* hereafter.

The following are not supported:

1. Cycles.

2. Parent pointers, or pointers from a node to higher levels of the tree.

3. Trees whose nodes can have an unbounded number of children.

A useful rule of thumb is whether every operation on the structure can be expressed as adding or removing a subtree as the child of a node by changing one pointer.

This definition encompasses a wide variety of structures, including all kinds of binary search trees, B-trees, stacks, queues, singly linked lists, and any other structures satisfying the preceding requirements.

2.3.3 Tree Representation

A tree must have a tree object containing its root. The root and all other nodes must be instances of an object supporting certain methods, which we call the **Node** object.

2.3.4 Indexing a Node's Children

The only restriction on the shape of a *Node* is that its possible children have an implicit indexing scheme beginning at zero and ending at one less than the maximum number of children. For example, a binary tree may use index 0 for the left child and 1 for the right child.

Beyond this, any internal implementation is acceptable so long as it supports the methods in the next section.

2.3.5 Required Node Methods

2.3.5.1 Minimum Requirements

The *Node* object must support at least:

1. A **copy constructor**, **copy-assignment operator**, and **destructor** (the rule of three).
2. **NodeType* getChild(int i)**, returning child number i.
3. **void setChild(int i, NodeType* n)**, setting child number i to n.
4. **NodeType** getChildPointer(int i)**, returning a pointer to the storage for child number i.
5. **static constexpr int maxChildren()**, returning the maximum number of children per node as a compile-time constant.

2.3.5.2 Search-Tree Methods

For search trees, implementing the following additional methods allows the library to provide useful extra operations automatically and to achieve better performance.

1. **bool hasKey(int key)**: returns whether the node contains `key`.
2. **bool traversalDone(int key)**: returns whether the search for `key` terminates at this node. This is useful when a key may appear multiple times, as in external search trees.
3. **int nextChild(int key)**: returns the child index along which to continue searching for the given key.
4. **int nextChild(NodeType* target)**: returns the child index along which to continue searching for a given target node.

2.4 Basic Concepts

2.4.1 Operations

An operation is a set of actions performed on a data structure, such as inserting a key-value pair into a map.

2.4.2 Read-Only Operations

Concurrent data structures implemented with the library allow read-only operations to be implemented *exactly as in their sequential versions*; consequently, they are not discussed individually. The sole restriction is that these operations traverse the tree exclusively from root to leaves.

The current implementation does not provide memory reclamation, so RCU read-side locking is unnecessary. In a future version with memory reclamation, a reader would simply initialize a lock object provided by the library.

2.4.3 The Original Tree

The original tree is the sequential tree defined by the user and satisfying the structural requirements described earlier.

2.4.4 Connection Point

The connection point is either the root or a node to which the copy tree will be attached as a child. For a particular operation, it is the node *below* which modifications will take place; everything above it remains unchanged. The child of this node to be replaced atomically is usually specified as well.

2.4.5 The Copy Tree

Every operation that modifies the concurrent structure is implemented by constructing a **new subtree** containing modified copies of original nodes and newly introduced nodes, together with references to original nodes. These are represented by a new tree structure whose elements are called *safe nodes*. The new structure **maintains the copy**

tree internally without attaching it to the original tree. If an operation finishes successfully, **the copy tree takes the place of the old subtree** by replacing a **child pointer** at the **connection point**.

Because the copy tree in this work is constructed exclusively using safe nodes, we also use "copy tree" to refer to the safe-node structure.

Constructing this tree ensures two things:

1. A reader can always traverse a valid subtree, because each copy is replaced only by another valid version.

2. The operation can be expressed as a tree mutation, with all changes recorded in the copy.

An important property of the copy tree is that its nodes may refer directly to nodes in the original tree as children, when those nodes will not be modified. Thus unchanged nodes need not be copied, but the copy tree can be expanded to include copies later if modifications become necessary.

The following illustration shows an internal copy tree constructed to modify a subtree. Nodes that will be replaced by copies are marked red. Note that both the copy and the original node E may be connected to node F.

2.4.6 Transaction

A transaction is a block of code that executes atomically.

2.4.7 Validation

Validation is the process performed inside a transaction to ensure that an operation completes atomically.

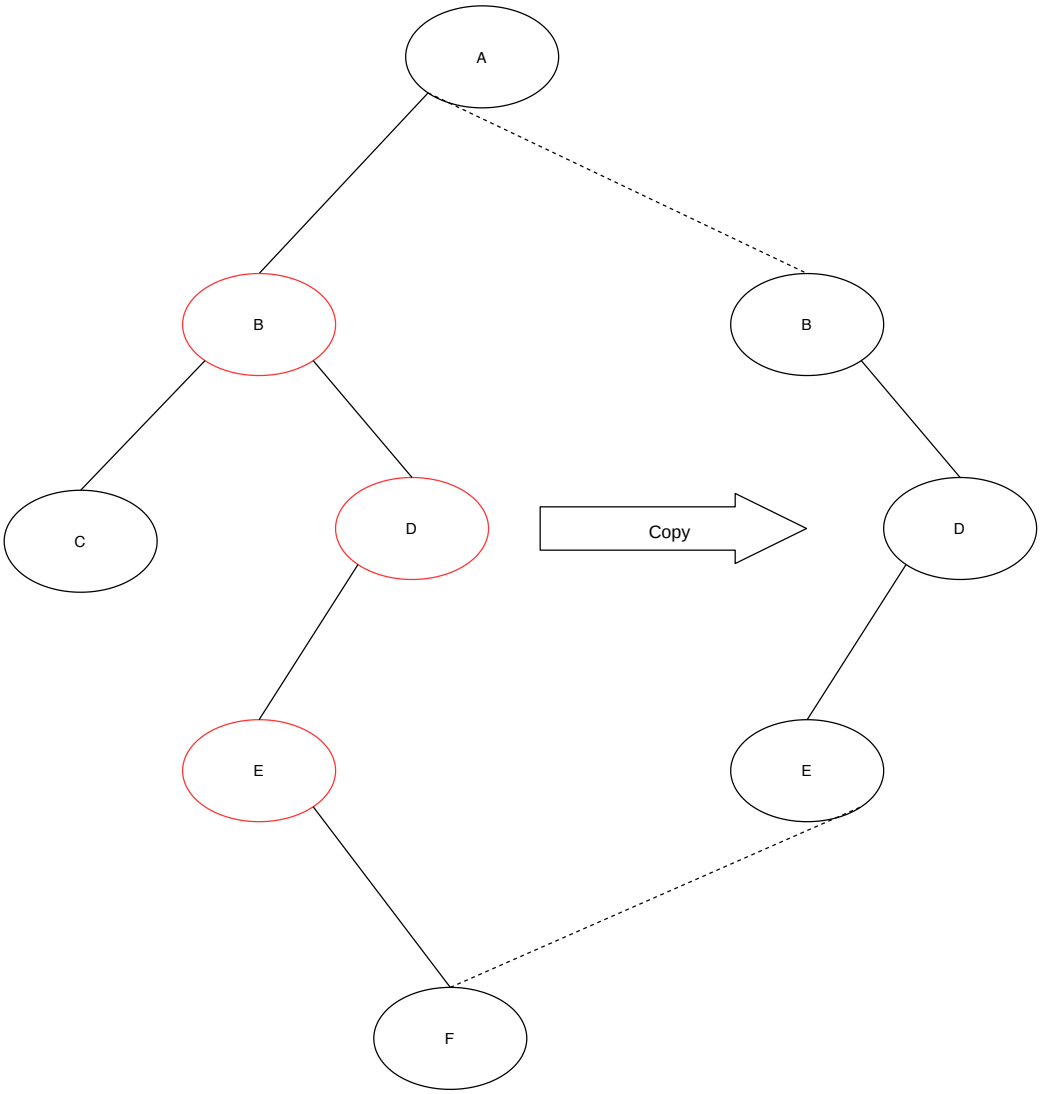


Figure 2.1: The Copy Tree

2.5 Core Library Components

2.5.1 Transaction Block

Every update operation on a concurrent structure must be enclosed in a transaction block. This block is essentially a macro responsible for restarting an operation if validation fails. Every operation to be parallelized by the library must be enclosed in such a block. The block is delimited by `TM_SAFE_OPERATION_START` and `TM_SAFE_OPERATION_END`.

Transaction parameters must also be configured: the number of retries before falling back to a global lock, and the statistics object. This is done by initializing a `Transaction` object.

Example:

```
// binary search tree insert method
bool insert(int key, int value) {

    int retries = 20;

    TM_SAFE_OPERATION_START {
        Transaction trans(retries, this->global_lock, this->stats);

        ...operation code...

    } TM_SAFE_OPERATION_END

    return true;
}
```

2.5.2 Connection-Point Search: Automatic and Path-Tracker

Every library operation begins by identifying the connection point and the child to be changed. The connection point is stored in a *ConnPointData* object, which holds information needed by the library. These fields will be examined in the technical analysis, but the most important is the *path stack*. The traversal path leading to the connection point is saved in a stack available through this object. The object can be constructed in two different ways, depending on whether the tree is a search tree or a general tree.

2.5.2.1 Search Trees

For a search tree, the library automatically provides the routine that finds an operation's connection point, using the node methods described earlier. Specifically, it provides:

```
ConnPointData<NodeType>
find_conn_point<NodeType>(int key, NodeType** root)
```

This function takes the node type as a template parameter and the required node's key as an argument. It uses **traversalDone(int key)** and **nextChild(int key)** to locate the connection point, returning the result in a `ConnPointData` object. *For search trees, the automatic methods are recommended because they enable performance optimizations and reduce opportunities for programming errors.*

2.5.2.2 General Tree Structures

To gather the required information and simplify programming, the library exposes a *PathTracker* abstraction. This object records the path taken through a tree and uses it to construct a *ConnPointData* object. It resembles an iterator that traverses a tree and points at one of its nodes. It provides the following methods:

1. `NodeType* getNode()` returns a pointer to the node at the current position.
2. `void moveToChild(int index)` moves the PathTracker to child `index` of the current node.
3. `void moveUp(int n_steps)` moves it `n_steps` steps toward the root.
4. `ConnPointData connectHere()` designates the current node as the connection point and returns a `ConnPointData` object.

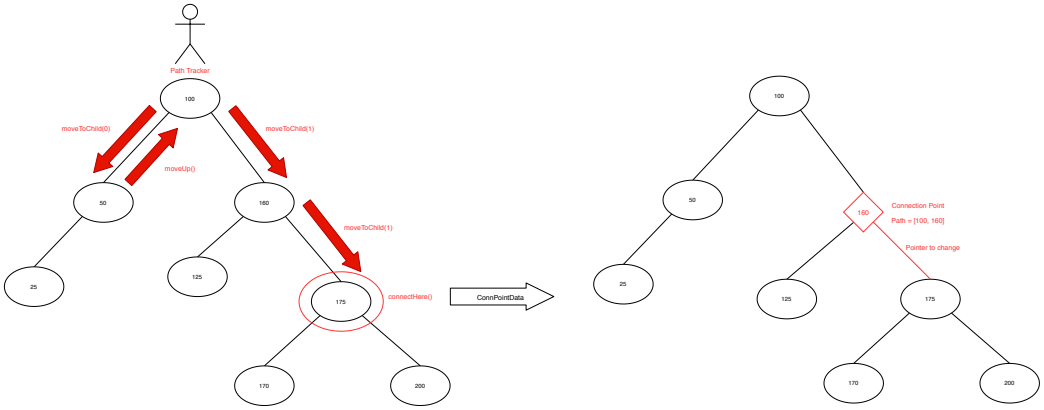


Figure 2.2: PathTracker

2.5.3 Connection-Point Search: Code Examples

2.5.3.1 Search in a General Tree

We show how to implement `find_conn_point` for the same search tree using `PathTracker`.

```
ConnPointData<TreeNode> find_conn_point_custom(int k, bool& found) {
    // initialize PathTracker with pointer to root
    PathTracker<TreeNode> tracker(&root);

    TreeNode* curr_node;
    int curr_key;

    // while tracker does not point to null
    while ((curr_node = tracker.getNode()))
    {
        // get the key
        // of the current node
        curr_key = curr_node->getKey();

         // if key is same as target
        if (curr_key == k) {

            found = true;   // declare it as found

            // return ConnPointData
            //with node as Connection Point
            return tracker.connectHere();

        } else if (k < curr_key) {
            tracker.moveToChild(0); // move left
        } else {
            tracker.moveToChild(1); // move right
        }
    }
    found = false;
    return tracker.connectHere();
}
```

We initialize the object with the address of the root pointer and traverse to the desired connection point. We then invoke `connectHere()`,

which returns a `ConnPointData` object containing the path stack and connection point.

2.5.3.2 Search in a Search Tree

As noted earlier, the search-tree case is straightforward:

```
// in the insert method of a bst

// find the connection point for inserting with key k
auto conn_point_snapshot = find_conn_point<TreeNode>(k,&root);

// if not found return false
if (conn_point_snapshot.found()) {
    return false;
}
```

2.5.4 SafeNode (Safe Node)

Safe nodes are the primary building block for constructing thread-safe tree structures with the library. They are objects parameterized by a node type through C++ templates; their type is `SafeNode<TreeNode>`, where `TreeNode` is the user-defined node class.

Safe nodes serve as the nodes of the copy tree and ensure that **every modification of the concurrent structure occurs atomically**. The technical analysis explains how this is achieved.

Safe nodes describe the changes to be made within a subtree. They provide methods similar to those of the original tree nodes. **Constructing a copy tree** with safe nodes ultimately **produces a subtree of user-defined nodes at the end of the operation**, without the hazards of concurrent mutation.

Importantly, the safe nodes *do not themselves become nodes in the concurrent structure*. They are created to manage changes during an operation and are retired when it succeeds or fails. A safe node acts as **a wrapper around an original tree node**, offering a small, convenient API for **describing changes to the tree**. Reading or modifying a SafeNode's children through its methods guarantees that **the data originate from a valid tree state** and that **the changes are applied atomically**. This requires only minor adjustments compared with sequential code.

Responsibilities of a Safe Node

1. Retaining references to the original tree node and its copy.
2. Managing connections among copies and to the original tree.
3. Recording modifications to the tree.
4. Creating copies lazily, only when a node is accessed for mutation (similar to copy-on-write).
5. Recording original child-pointer values when copies are created, for later *validation*.

2.5.4.1 Supported Methods

1. **SafeNode<NodeType>* getChild(const int index)** returns the specified child of the wrapped node, itself wrapped in a safe node. It therefore performs implicit wrapping (see `ConnPoint`). If the child was accessed before, the stored safe node is returned directly; otherwise a new safe node is constructed around the original child. It returns `nullptr` when no child exists.

2. **SafeNode<NodeType>* peekChild(const int index)** returns a reference to the original node's child at the specified index without wrapping it in a safe node. It is useful when only the child node's address is needed. The library guarantees that, if validation succeeds, the node continues to have that child until the transaction ends, but provides no guarantees concerning the grandchildren of the referenced child.

3. **SafeNode<NodeType>* setChild(const int index, SafeNode<NodeType>* newChild)** sets `newChild` as the child at `index`. It becomes visible through future `getChild` calls, and the original nodes will also be connected in the copy tree.

4. **NodeType* rwRef()** returns a safe reference: a pointer to a copy of the wrapped node, creating that copy if necessary, through which its data may be modified in a thread-safe manner. These changes are discarded if the transaction fails validation.

5. **NodeType* peekOriginal()** returns an unsafe reference to the original node.

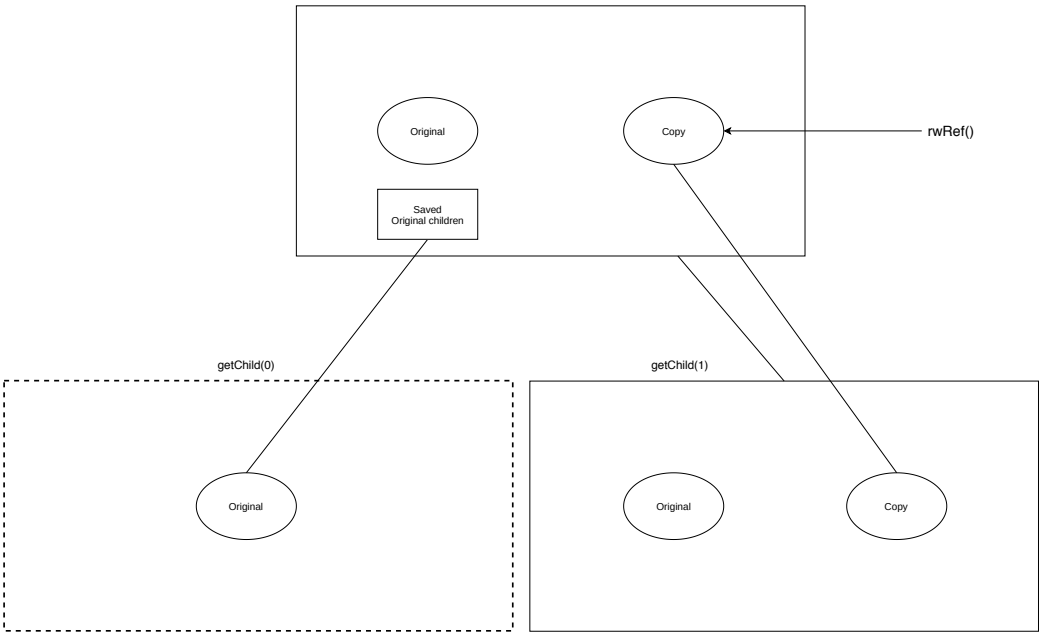


Figure 2.3: Safe Node: First Access (Left) and Repeated Access to a Child

2.5.4.2 Using Safe Nodes

To modify a node, we first access it starting from the copy tree's root, invoking *getChild*. The copy tree root, itself a safe node, is provided automatically by the *getRoot()* method of *ConnPoint*; other safe nodes are constructed lazily during traversal. Two cases may then arise:

1. We want to modify **the node's children**, for example, to assign a new child. We wrap that child in a safe node (using *wrapSafe()*, *createSafe()*, etc.) and attach it with *setChild*.

2. We want to modify **the node's internal data**. We obtain a safe reference via *rwRef()* and modify the data just as in sequential code.

2.5.5 SafeNode: Code Examples

We illustrate safe-node methods through a few representative use cases.

2.5.5.1 Example 1: Swap the Left and Right Children of a Binary-Tree Node

```
void swapLeftRight(SafeNode<MyNode>* sample_node) {
    auto left_child = sample_node->getChild(0); // read left child
    auto right_child = sample_node->getChild(1); // read right child

    sample_node->setChild(0, right_child); // set right as left
    sample_node->setChild(1, left_child);  // set left as right
}
```

The code is strikingly similar to its sequential equivalent and is self-explanatory, but the library ensures thread safety. Note that all connections use safe nodes exclusively.

2.5.5.2 Example 2: Update a Value in a Key-Value Tree Node

```
void updateValue(SafeNode<MyNode>* sample_node, ValueType new_value) {
    MyNode* sample_node_ref = sample_node->rwRef(); // get a rw reference
    sample_node_ref->value = new_value;
}
```

Again, the code closely resembles the sequential version, and mutation is performed through a safe reference.

2.5.5.3 Example 3: Swap Child Nodes if Their Keys Sum to an Even Number

```
void swapSumEven(SafeNode<MyNode>* target_node) {
    auto left_key = target_node->peekChild(0)->key;
    auto right_key = target_node->peekChild(0)->key;

    if ((left_key + right_key) % 2 == 0) {
        swapLeftRight(target_node);
    }
}
```

Because we do not know in advance whether the tree needs to change, we first use *peekChild* to test the condition. If it holds, we call *swapLeftRight*, which creates copies when it invokes *getChild*.

2.5.6 ConnPoint (Connection Point)

The `ConnPoint` object holds the root of the copy tree and the node at which that tree will be installed (the connection point). It is responsible for validation and for the final installation of the copy tree into the original tree. It is always constructed inside a transaction block, with `ConnPointData` and `Transaction` as arguments.

Its first, fundamental role is **to maintain the root of the copy tree**. This root will be attached as a child of the connection point (or installed as the original tree's new root). Three methods are provided:

1. `SafeNode<NodeType>* getRoot()` has two behaviors. *If no root is defined*, it wraps the connection point's child to be modified in a safe node, sets it as the copy tree root, and returns it. *If a root already exists*, it simply returns it.

2. `SafeNode<NodeType>* setRoot(SafeNode<NodeType>* new_root)` sets a safe node as the copy tree's new root and returns it for convenience.

3. `SafeNode<NodeType>* newTree(NodeType* new_root)` wraps a newly created node in a safe node and installs it as the root of the copy tree.

These methods initialize the copy tree, allowing the entire subtree beneath the connection point to be modified through it.

When do changes become visible in the original tree? Validation and installation of the new subtree (containing the user nodes held by the safe nodes of the copy tree) take place when the `ConnPoint` object's destructor is invoked.

2.5.6.1 Creating Safe Nodes

The second major use of `ConnPoint` is wrapping ordinary node objects in safe nodes. This process is called **wrapping**. Wrapping an original tree node adds it to the **validation set** of the `ConnPoint`. The validation set ensures that **an operation will not commit if any of the stored nodes' child pointers change during the operation**. This property provides thread safety and must hold for every modification. Consequently, every modification to a node's child pointers must be performed through safe-node methods.

There are two general ways to wrap a node: indirectly through safe-node methods, or *directly* through `ConnPoint` methods. Three methods are available:

1. `SafeNode<NodeType>* wrap_safe(NodeType* some_node)` wraps a node and **adds it to the validation set** of the `ConnPoint`. Any nodes *wrapped indirectly* via that safe node's `getChild` method are **also added to the validation set**.

2. `SafeNode<NodeType>* create_safe(NodeType* some_node)` wraps a node **without adding it to the validation set**. It is used for newly created nodes not yet part of the tree, which therefore need no validation. Descendants *wrapped indirectly* through `getChild` are also **excluded from the validation set**.

3. `SafeNode<NodeType>* wrap_no_validate` behaves like `create_safe` but **marks the wrapped nodes as belonging to the original tree rather than being newly allocated**; therefore they must not be deleted if validation fails.

2.5.6.2 Moving Toward the Root

In some situations, **the eventual connection point cannot be determined in advance**. For example, inserting into a balanced tree may require changes that begin deep in the tree and propagate toward the root. To support this essential pattern, `ConnPoint` provides `SafeNode<NodeType>* popPath()`.

This method uses the **path stack** constructed while locating the initial connection point to **make its parent the new connection point**. At the same time, it **wraps the former connection point as the new root of the copy tree** and **joins it to the existing copy tree**. The effect is as if the initial search had chosen the parent as the connection point.

This enables modifications to nodes **above the original connection point**, while maintaining the rule that all changes occur beneath the new connection point. The method returns the new root of the copy tree (the previous connection point), wrapped in a safe node, or `nullptr` if no parent exists. In that case, the copy tree is installed as the root of the original tree.

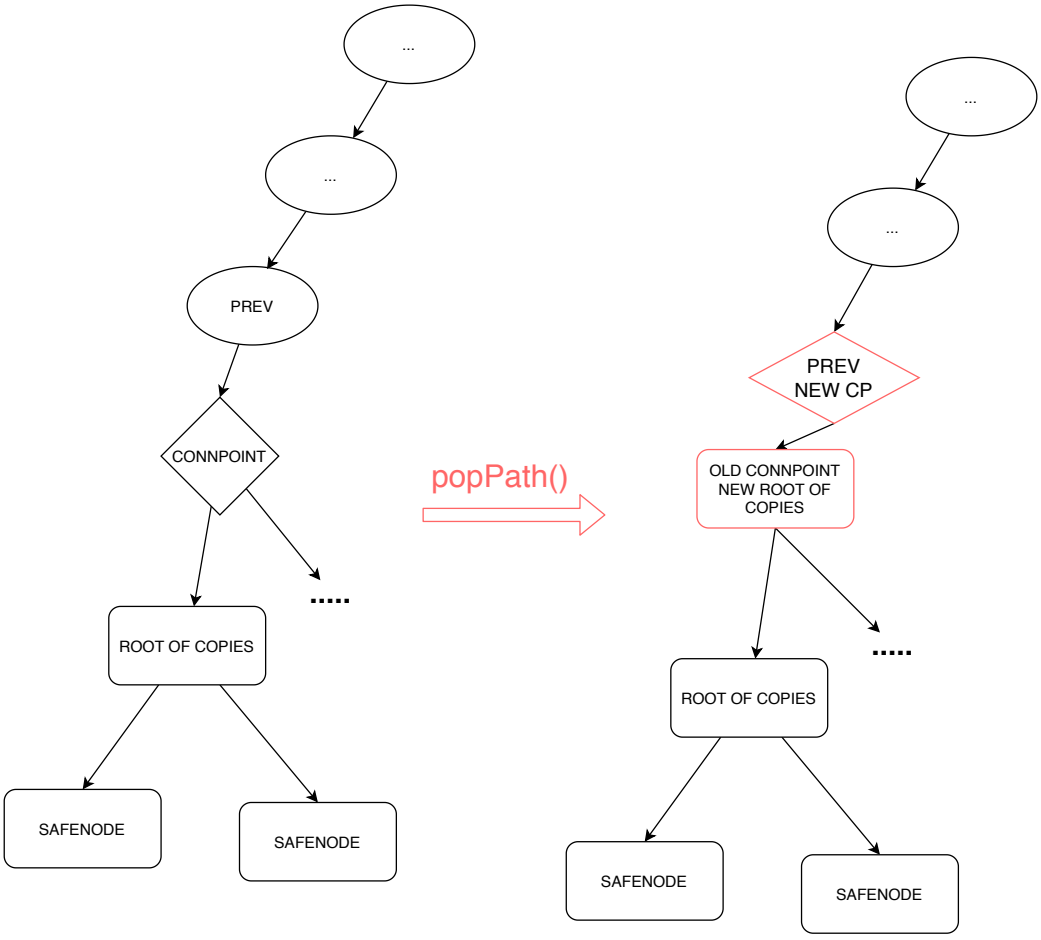


Figure 2.4: Moving the Connection Point Toward the Root

2.5.7 ConnPoint: Code Examples

2.5.7.1 Initializing a Connection Point

```
// skeleton of every operation
TM_SAFE_OPERATION_START {

    // init transaction
    TSX::Transaction t(retries,_lock, stats[t_id]);

    // find connection point
    auto conn_point_snapshot = find_conn_point<TreeNode>(k,&root);

    // initialize ConnPoint with required objects
    ConnPoint<TreeNode> conn(conn_point_snapshot, t);

    // do operations using ConnPoint
    ..............
}
```

Every operation implemented using the library has this overall structure.

2.5.7.2 Reading and Installing a New Root

```
// in transaction block
{
    // skeleton here
    ...

    // conn is the ConnPoint object

    SafeNode<NodeType>* curr_root = conn.getRoot();

    // SafeNode new_root will be installed if there's no root

    if (!curr_root) {
        conn.setRoot(new_root);
    }
}
```

At the end of this operation, the `ConnPoint` destructor is called. If the connection point initially had no child, the node wrapped in `new_root` is installed; otherwise, nothing changes.

2.5.7.3 Tree Rebalancing with popPath

As noted earlier, rebalancing is a good example of `popPath()`. First we find the connection point, attach the new node as its child, and make that new node the root (and only node) of the copy tree. However, insertion may have disturbed the tree's balance. Therefore:

- Extend the copy tree by making the current connection point its new root, and designate that point's parent as the next connection point.
- Apply a rotation to rebalance the tree at this level. The `rebalance_ins` routine uses safe nodes and may further extend the copy tree with any resulting changes.
- Make the rotation's result the new root.

Repeating this process until we reach the original root or restore balance constructs a copy tree containing all required modifications and identifies the connection point at which to attach it.

2.5.7.4 Code Example

```
// inserting in AVL Tree

// in skeleton
...

// conn is the ConnPoint object

// new node will be added below connection point
conn.setRoot(conn.wrap_safe(new AVLNode(...));

// but tree could be unbalanced now
// let's rebalance it

// keep setting parent as new connection point as long as it is
//unbalanced

for (SafeNode<TreeNode>* n = conn.popPath(); n != nullptr; n = conn.popPath()
    // get a reference to read data
    auto n_data = n->rwRef();
    int height_old = n_data->height;

    // do a rebalance of the current node
    n = rebalance_insert(n, k , rotation_happened);

    // set the result of the rebalancing as the new root
    // (to be inserted below the current connection point)
    conn.setRoot(n);    // change the root of the
                        // tree of copies to make the change visible

    // if balanced, break
    if (height_old == n_data->height && !rotation_happened) {
        break;
    }
    ...
```

2.6 Summary

To parallelize an operation on a tree data structure:

1. Create a transaction block.
2. Locate and designate the connection point, automatically or manually, and initialize a `ConnPoint` object.
3. Initialize the copy tree using `ConnPoint` and access its root through *getRoot()*.
4. Convert functions and methods to use safe nodes by changing their types. The safe-node tree automatically expands toward children or toward the root as nodes are accessed. Additions are performed by wrapping nodes in safe nodes and attaching them with `setChild`.
5. The generated copy tree is attached automatically when the transaction succeeds.

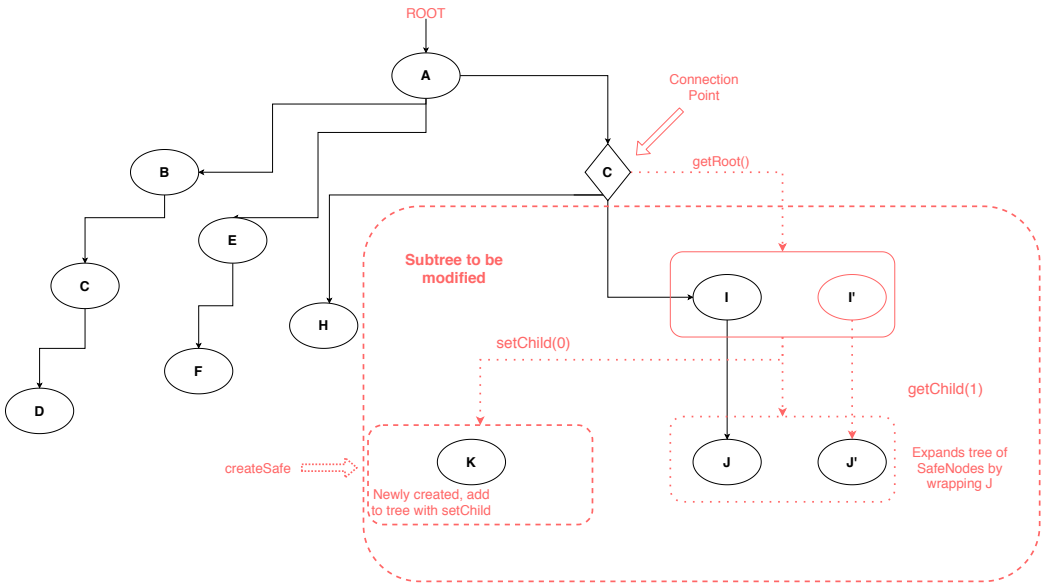


Figure 2.5: The Parallelization Library

2.7 Parallelizing an AVL-Tree-Based Map

2.7.1 Introduction

In this section we demonstrate how to parallelize an AVL search tree by presenting its most important operations. An AVL tree is an ideal example because it uses all the tools offered by the library. Minor details concerning helper methods are omitted; the complete implementation is available through the references at the end of the thesis.

2.7.2 Required Includes

All required library methods and objects are defined in the `SafeTree.hpp` header, in the `TSX` namespace.

2.7.3 The Node Object: AVLNode

Below is a minimal implementation of an AVL-tree node, which is itself a tree. The methods are simple and mostly self-explanatory.
Only `traversalDone` and `nextChild` need special attention. `traversalDone` returns `true` when the searched key matches the node's key. This rules out duplicate keys, since a search stops at the first match. That is precisely the purpose of the method: to stop traversal whenever it should terminate. If duplicates were supported, the condition would differ; for example, this method might always return `false`.

The library user must verify that these three methods work correctly in the sequential tree before attempting parallelization. Incorrect behavior can compromise validation and introduce concurrency bugs. Note also that all library methods belong to the `TSX` namespace, which has been omitted here for brevity (equivalent to `using namespace TSX`).

```
class AVLNode {
    friend class AVLTree<ValueType>;
    private:
        int key;
        ValueType value;
        AVLNode* children[2];
        int height;
    public:
        AVLNode(int key, ValueType val, AVLNode* left_child, AVLNode* right_c
            children[0] = left_child;
```

```
    children[1] = right_child;
}

// to satisfy search tree interface
bool hasKey(int key_requested) {
    return key == key_requested;
}

int nextChild(int desired_key) const {
    return desired_key < key ? 0 : 1;
}

bool traversalDone(int desired_key) const {
    return key == desired_key;
}

int nextChild(const AVLNode* target) const {
    return target->key < key ? 0 : 1;
}

// and the basic required methods
 static constexpr int maxChildren() {
    return 2;
}

AVLNode** getChildPointer(int i) {
    assert(i >= 0 && i < 2);
    return &children[i];
}

AVLNode* getChild(int i) {
    assert(i >= 0 && i < 2);
    return children[i];
}


void setChild(int i, AVLNode* node) {
    assert(i >= 0 && i < 2);
    children[i] = node;
}
```

2.7.4 The Tree Object: AVLTree

Our tree only needs a root, an instance of the library's `SpinLock`, and per-thread statistics objects. Statistics provide useful information about failed transactions, helping diagnose performance problems and tune the retry count. We also define `TreeNode` as a shorthand for `AVLNode<ValueType>` to save space in the printed thesis.

```
template <class ValueType>
class AVLTree {
    private:
        AVLNode<ValueType>* root;

        // tree global lock
        TSX::SpinLock &_lock;

        // transactional retries
        const int trans_retries = 30;

        // transactional stats objects
        constexpr int THREAD_AMOUNT_MAX = 100;
        TSX::TSXStats stats[THREAD_AMOUNT_MAX];

        // alias for AVLNode
        using TreeNode = AVLNode<ValueType>;
}
```

2.7.5 Lookup

Lookup for a node in the tree is automatically provided by the library through its `find` method:

```
TreeNode lookup(int desired_key) {
    return find<TreeNode>(root,desired_key);
}
```

`find` returns either the node holding the desired key or `nullptr` if none exists. Internally it uses the node methods described earlier. This lookup is lock-free and guarantees progress because it traverses the tree just as a sequential lookup does.

2.7.6 Insertion: Adding a Node

The first step in implementing insertion is to use the standard operation skeleton. Note that we take the thread's index as a parameter in order to identify the corresponding statistics object.
A variable must specify the maximum number of HTM transaction attempts before using the lock.

```
bool insert(const int k, ValueType val, int t_id) {
    // needs to be mutable
    int retries = trans_retries;

    TM_SAFE_OPERATION_START {

        TSX::Transaction t(retries,_lock, stats[t_id]);

        /* FIND PHASE */

        auto conn_point_snapshot = find_conn_point<TreeNode>(k,&root);

        if (conn_point_snapshot.found()) {
            return false;
        }
            /* FIND PHASE END */

        ConnPoint<TreeNode> conn(conn_point_snapshot, t);

        ...
        //insertion code goes here
        ...
    } TM_SAFE_OPERATION_END

    return true;
}
```

At this point, the method has either found a node with key k and returned *false*, or found the connection point, the parent under which the new node should be inserted, and initialized the copy tree via a *ConnPoint* object. Initially, the change to describe is simply attaching a new node as a child of the connection point. This is accomplished by wrapping the newly allocated node in a safe node and making it the root of the copy tree!

```
...
ConnPoint<TreeNode> conn(conn_point_snapshot, t);

// wrap newly inserted node
auto node_to_be_inserted = conn.create_safe(
                            new AVLNode<ValueType>(k,val,nullptr,nullptr)
                            );
// insert it
conn.setRoot(node_to_be_inserted);

...
//balancing code
...

} TM_SAFE_OPERATION_END
```

If we were implementing an unbalanced binary search tree, the code would end here. The library would ensure that insertion occurs atomically. An AVL tree, however, may no longer satisfy its balance invariant after insertion.

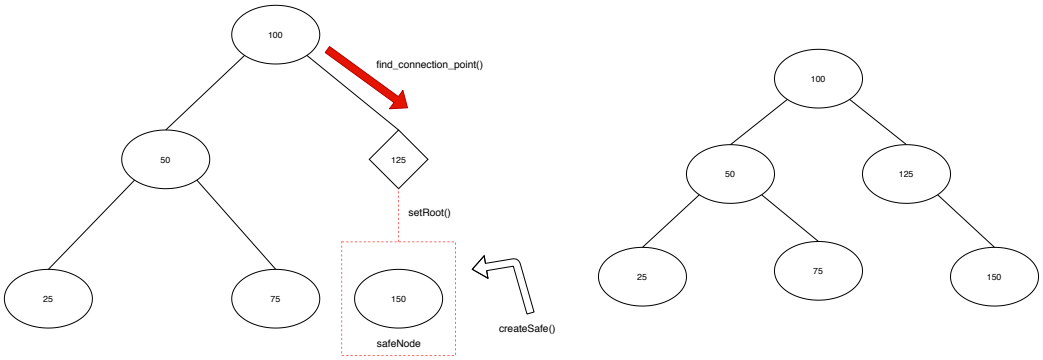


Figure 2.6: Node Insertion

2.7.7 Insertion: Rebalancing a Sequential Tree

In the previous section, we reached the point where we have a connection point and a copy tree containing a new node at its root. Before using the library to implement rebalancing, let us examine the sequential version.

Assume that we saved the path from the root to the parent of the newly inserted child in a stack named `insertStack`. At each tree level, we must perform any necessary *rotations* and connect the resulting subtree to the node at the next level. At the same time, we test the condition indicating that the tree is no longer changing, so that we can stop before reaching the root if balance has already been restored.

```
auto rotationHappened = false;

AVLNode *prev, *n;
unsigned int heightOld = 0;

for (n = insertStack.pop() , prev = new_node; n; prev = n, n = insertStack.pop()) {
    //insert new node or rebalanced node
    if (rotationHappened) {
        if (k < n->key) {
            n->setL(prev);
            rotationHappened = false;
        } else {
            n->setR(prev);
            rotationHappened = false;
        }
    }

    n = rebalance_ins(n,k,rotation_happened);

    if (height_old == n->height && !rotation_happened) {
        break;
    }
}
```

2.7.7.1 The Sequential Rebalancing Method

The `rebalance_insert` method determines where a rotation is required, performs it, and returns the new root of the level at which it was called. A typical implementation for our AVL tree follows.

```
SafeNode<NodeType> rebalance_insert(SafeNode<NodeType> node) {
    heightOld = n->height;

    // calculate node height
    n->height = maxheight(n->getChild(0), n->getChild(1)) + 1;

    // calculate node's balance
    auto balance = nodeBalance(n);

    rotation_happened = true;

    if (balance > 1 && k < n->getChild(0)->key) { // right rotate
        n = rightRotate(n);
    } else if (balance < -1 && k > n->getChild(1)->key) { //left rotate
        n = leftRotate(n);
    } else if (balance > 1 && k > n->getChild(0)->key) { // left right rotat
        n->setL(leftRotate(n->getChild(0)));
        n->height = maxheight(n->getChild(0), n->getChild(1)) + 1;
        n = rightRotate(n);
    } else if (balance < -1 && k < n->getChild(1)->key) { // right Left Rota
        n->setR(rightRotate(n->getChild(1)));
        n->height = maxheight(n->getChild(0), n->getChild(1)) + 1;
        n = leftRotate(n);
    } else {
        rotation_happened = false;
    }

    n->height = maxheight(n->getChild(0), n->getChild(1)) + 1;
}
```

We also need to show one of the rotations so we can explain its conversion into a concurrent version. The right rotation is used as an example.

```
static AVLNode* rightRotate(AVLNode *z) {
        // left becomes root
        auto newRoot = z->getChild(0);

        // left's right will be moved to old root's left
        auto T3 = newRoot->getChild(1);

        // connect old root right of new root
        newRoot->setR(z);

        // connect left's right
        z->setL(T3);

        z->height = maxheight(z->getChild(0), z->getChild(1)) + 1;

        newRoot->height = maxheight((newRoot->getChild(0)), newRoot->getChild(1)) +

        return newRoot;
    };
```

2.7.8 Insertion: Rebalancing Function for the Concurrent Tree

What changes are required to make rebalancing part of a concurrent insertion operation? We begin with the rule:

- For changes to node connections, use safe nodes.
- For changes to node data, use safe references.

Let us work backward and convert the right rotation using this rule:

```
static SafeNode<AVLNodeType>* right_rotate(SafeNode<AVLNodeType>* z) {
    /*  T1, T2, T3 and T4 are subtrees.
        z                                      y
        / \                                  /   \
```

```
    y   T4      Right Rotate (z)          x      z
    /   \          - - - - - - - - ->    /  \    /  \
    x    T3                             T1  T2  T3  T4
    / \
    T1   T2 */

    // left becomes root
    auto newRoot = z->getChild(0);  // use the SafeNode to get children

    // left's right will be moved to old root's left
    auto T2 = newRoot->getChild(1);

    // connect old root right of new root
    newRoot->setChild(1,z);

    // connect left's right
    z->setChild(0,T2);

    // copy to change
    auto z_safe = z->rwRef();   // for values get a reference
    auto newRoot_safe = newRoot->rwRef();

    // set height with references
    z_safe->height = max_height(z_safe->getChild(0), z_safe->->getChild(1)) +

    // max height returns the max height of the two trees and zero for empt
    newRoot_safe->height =
        max_height(newRoot_safe->getChild(0),
            newRoot_safe->getChild(1)) + 1;

    return newRoot;
}
```

As we can see, the code is very similar to its sequential counterpart. The changes are that the function consumes and produces safe nodes, affecting only the copy tree (and hence remaining thread-safe). We use `getChild` and `setChild` to change connections, keeping the code structurally identical to the sequential version, and safe references to update node heights. The user need not manage the way the copy tree changes: it is expanded and modified automatically. This ease of use is precisely what the library aims to achieve.

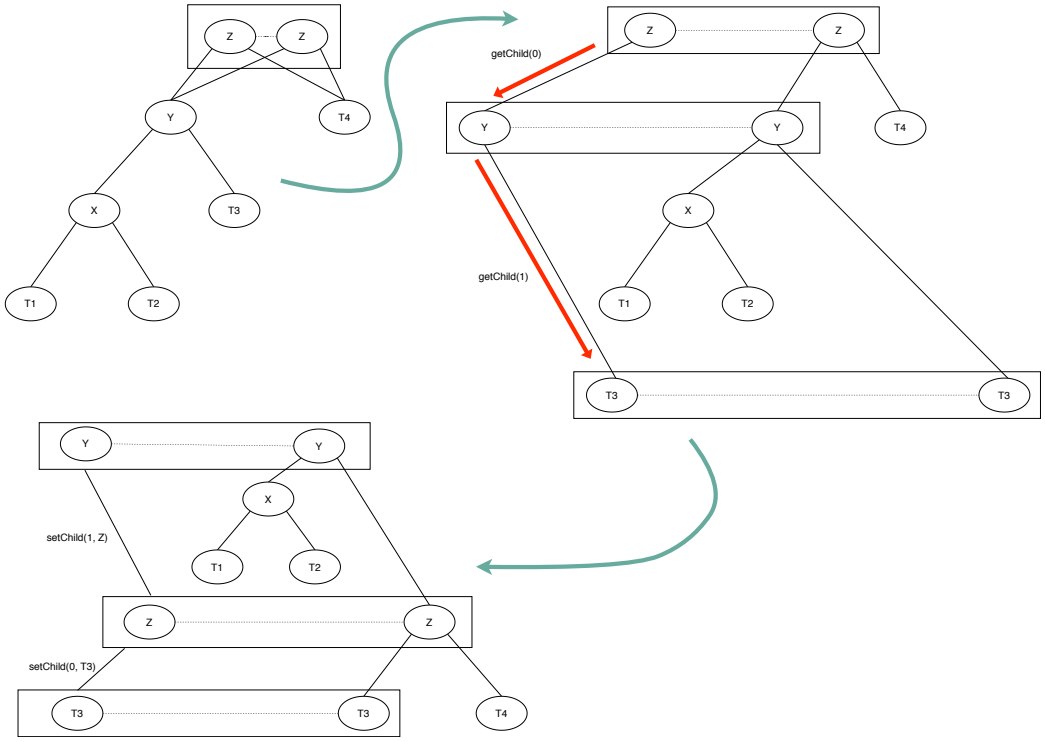


Figure 2.7: Library-Based Right Rotation

We apply the same transformation to the rebalancing method so that it operates on the copy tree.

```
// apply rebalancing for an insertion operation
SafeNode<TreeNode>* rebalance_ins(SafeNode<TreeNode>* n,
    int k, bool& rotation_happened) {
     // reads and writes are safe
    auto n_data = n->rwRef();
    // calculate node height
    n_data->height =
        max_height(n_data->getChild(0), n_data->getChild(1)) + 1;
    // calculate node's balance
    auto balance = node_balance(n_data);
    rotation_happened = true;
    // for tree modifications (change children etc) use SafeNode
    if (balance > 1 && k < n_data->getChild(0)->key) {
        // right rotate
        n = right_rotate(n);
    } else if (balance < -1 && k > n_data->getChild(1)->key) {
        //left rotate
        n = left_rotate(n);
    } else if (balance > 1 && k > n_data->getChild(0)->key) {
        // left right rotate
        n->setChild(0 ,left_rotate(n->getChild(0)));
        n_data->height =
            max_height(n_data->getChild(0), n_data->getChild(1)) + 1;
        n = right_rotate(n);
    } else if (balance < -1 && k < n_data->getChild(1)->key) {
        // right Left Rotate
        n->setChild(1,right_rotate(n->getChild(1)));
        n_data->height =
            max_height(n_data->getChild(0), n_data->getChild(1)) + 1;
        n = left_rotate(n);
    } else {
        rotation_happened = false;
    }
    // calculate new height
    n_data->height =
        max_height(n_data->getChild(0), n_data->getChild(1)) + 1;
    return n;
}
```

2.7.9 Insertion: Rebalancing the Concurrent Tree

Now that we have a function that rebalances each level of the copy tree, we only need to supply the right levels and connect them correctly. The library's saved path stack greatly simplifies the code. The rebalancing algorithm described earlier reduces to these steps:

1. Move the connection point up one level, making the previous connection point the new root of the copy tree.
2. Rebalance the copy tree and calculate its new root.
3. Set that result as the copy tree's root.

The first step introduces higher-level nodes into the copy tree so they can be rebalanced. But which child of the new connection point should receive the copy tree when insertion completes? During traversal, the path stack stores each node along with the index of the child taken next. Thus the copy tree is attached automatically in that position. In this search tree, the index comes from the node's `next_child` method, so no additional initial check is needed: we simply attach the root. If the copy tree needed to be attached elsewhere, we could move up another level, include the connection point in the copy tree, and use `setChild` to select the correct child.

```
// go up path and rebalance
for (SafeNode<TreeNode>* n = conn.popPath(); n != nullptr; n = conn.popPath()) {
    auto n_data = n->rwRef();
    int height_old = n_data->height;

    n = rebalance_ins(n, k , rotation_happened);
    // apply rebalancing to all
    // required nodes
    // rotation returns the new root to place
    // below the connection point

    conn.setRoot(n);     // change the root of the
                         // tree of copies to make the change visible
    if (height_old == n_data->height && !rotation_happened) {
        break;
    }

}
```

The concurrent code is **almost identical** to the sequential version, making conversion extremely easy. It is enough to **expand the safe-node tree to each node that must be accessed** and **use safe nodes and references to express modifications**.

Indeed, simply **changing node types to safe-node types** is sufficient to transform complex methods such as rotations.

Furthermore, the library's search-tree support means it can **use the supplied traversal methods to simplify the programmer's work**. For example, calling `setRoot` is enough to attach the new copy-tree root automatically at the right location, since the library knows the connection point and which child must change.

2.8 Debugging

Debugging is essential when constructing concurrent data structures: their correctness and efficiency must be verified, and logical errors addressed. Let us examine how the library helps.

2.8.1 Advantages of the Library

1. **User-provided methods**. Users write only sequential methods, regardless of how the library uses them. If they behave correctly sequentially, they will behave correctly when invoked by the concurrent implementation. Much of debugging therefore becomes sequential debugging.

2. **Changes to local copies**. All user-written code modifies copies of subtrees private to each thread. Users can print the changes and inspect the state of such a copy without worrying that another thread is simultaneously changing it; they simply use safe references.

3. **Centralized synchronization logic**. The library itself can, of course, have bugs that cause incorrect concurrent behavior. Fixing those bugs improves the library, so each additional data-structure implementation tests or improves the correctness of all the others.

2.8.2 Common Mistakes

This section describes several common mistakes when building structures with the library and suggests how to avoid them.

Accessing a node's children without a safe node Thread safety depends on validating that the child pointers of every node in the copy tree remain unchanged from the beginning to the end of the transaction. This is guaranteed only by wrapping the nodes in safe nodes. All access to children must therefore go through safe nodes and their methods.

Modifying node data without a safe reference Mutating data through an original node rather than a safe reference leads to undefined behavior.

Synchronization errors arising from sequential methods Beyond the two issues above, particularly with search trees, the user's sequential methods must be completely correct. Otherwise tree parallelization can fail: for example, an incorrect `nextChild` implementation can prevent proper validation.

Calling getChild or setChild on original nodes Safe-node methods intentionally share names with the corresponding methods on original nodes to simplify conversion. We recommend changing variable types to safe-node types when using safe methods, to avoid inadvertently invoking the original sequential versions.

Chapter 3

Technical Analysis of the Library

3.1 Overview and Required Methods

To be as general-purpose as possible, the library relies extensively on the user's implementation. It uses C++ templates so that each exposed object accepts a user-defined node type as a parameter, generating *monomorphic* code specialized for that type.

This allows all library classes to inline the node object's methods. When used carefully, the library can therefore be regarded as a zero-cost abstraction compared with manually implementing RCU-HTM.

The methods described in the previous chapter must be provided so that the library can manipulate original-node links when attaching copies, access nodes' children, and determine traversal rules for search trees.

3.2 Synchronization

3.2.1 SpinLock

A new spinlock was developed for this library so that its internal state can be inspected inside an HTM transaction. Standard-library implementations conceal this information because it is not safely accessible by arbitrary threads. Our implementation uses C++ atomic variables.

```
class SpinLock {
    private:
        enum {
            UNLOCKED = false,
            LOCKED = true
        };
        std::atomic<bool> spin_lock_;
    public:
        SpinLock(): spin_lock_(false) {}
        void lock() noexcept{
            for (;;) {
                // test
                while (spin_lock_.load(std::memory_order_relaxed) == LOCKED)
                if (!spin_lock_.exchange(LOCKED, std::memory_order_acquire))
                    break;
                }
            }
        }
        void unlock() noexcept{
            spin_lock_.store(false, std::memory_order_release);
        }
        bool isLocked() noexcept {
            return spin_lock_.load(std::memory_order_relaxed);
        }
};
```

This is a conventional implementation in which the atomic `exchange` instruction guarantees atomic lock acquisition. It uses TTAS (test-and-test-and-set) to avoid unnecessary polling with atomic instructions, and the `_mm_pause()` intrinsic to reduce wasteful spinning by introducing short delays. Although the latter optimization might appear counterproductive, it improves performance by allowing the memory location time to change state before the next polling attempt.

3.2.2 TSXGuard

The central library component that enables thread-safe execution is called *TSXGuard*.
It is an object whose constructor begins a hardware-memory transaction and which also manages transaction aborts and completion.
The simplest provided version is `TSXGuardWithStats`. An instance is created inside a loop controlled by a Boolean variable. It is initialized as follows:

```
TSXGuardWithStats(const int max_tx_retries,
            SpinLock &mutex,
            unsigned char &err_status,
            TSXStats &stats,
            bool disabled = false,
            RETRY_STRATEGY strat = STUBBORN)
```

The parameters are:

- `max_tx_retries`: the maximum number of aborts before falling back to locking.
- `mutex`: a reference to the lock.
- `&err_status`: a reference to the loop condition, so the loop terminates after a successful transaction.
- `&stats`: a reference to a statistics object.
- `disabled`: a flag that disables the mechanism.
- `strat`: a retry strategy; `STUBBORN` counts each abort as one retry, while `HALF` halves the retries remaining after each abort.

Internally, the constructor begins an HTM transaction or acquires the `SpinLock` if retries have been exhausted. Thus, after construction, each subsequent instruction executes either within a transaction or while holding the lock, and is therefore atomic with respect to other protected operations.
When does the transaction finish? When the object goes out of scope, its destructor commits the transaction or releases the lock. What happens on an abort? Control returns to the constructor, which records the abort in the statistics, decreases the remaining retry count, and tries again.

3.2.2.1 Usage Example

```
TSXStats stats;

bool transaction_complete = false;

const int retries = 20;

SpinLock lock;

while (!transaction_complete) {
    TSXGuardWithStats guard(retries, lock, transaction_complete);

    // any code in this block will run atomically

}
```

This object therefore provides a mutually exclusive region whose execution is thread-safe.

3.2.3 TSXTransOnlyGuard

Internally the library uses the slightly different `TSXTransOnlyGuard` object. It is used in the same way and behaves similarly, but does not manage the fallback lock. Lock management is delegated to the public API's `Transaction` object, allowing the user to supply their own lock; `TSXTransOnlyGuard` is an internal implementation detail.

3.2.4 Transaction

The `Transaction` object simply acquires the `SpinLock` when retries are exhausted and releases it in its destructor if it was acquired.

3.2.5 The Transaction Block

A transaction block is delimited by the macros `TM_SAFE_OPERATION_START` and `TM_SAFE_OPERATION_END`. The start macro hides a loop that implements the retry logic. It expands to:

```
__internal__thread_transaction_success_flag__ = false;
while(!__internal__thread_transaction_success_flag__)
```

As shown, an internal condition variable serves as `err_status` for `TSXTransOnlyGuard`, and the macro begins a loop testing this condition. Consequently it must always be followed by a braced code block. The end macro is normally only syntactic decoration and expands to nothing. However, it also supports an additional operating mode, *EARLY ABORT*.

3.3 Search

3.3.1 ConnPointData

A `ConnPointData` object is the result of searching for an operation's connection point and is necessary to initialize the copy tree (discussed next).

Besides the connection point's address, it must include:

1. The path followed to the connection point.
2. The address of the pointer holding the root of the original tree (so the root itself can be replaced).
3. The index and address of the child to be changed.
4. Whether that child was found.

`ConnPointData` contains all this information and is constructed automatically using one of the methods below.

3.3.2 Automatic Search Functions

As noted earlier, when the library is used to build a search tree, it automatically provides `find_conn_point`. Its template parameter is the node type; its arguments are the search key and the address of the original tree's root pointer.

Let us examine its implementation:

First, it reads the root through the supplied pointer. It does so only once because the root may change while the method is executing.

```
// find_conn_point: traverse tree with given root
// and return a ConnPointData object for the
// node with the given key. The node will be determined
// by the traversalDone method
//and the path taken by the nextChild method.

template <class NodeType>
ConnPointData<NodeType> find_conn_point(int key, NodeType** root) {
    // everything we need to create a conn point
    ConnPointData<NodeType> result;

    result.root_of_structure = root;

    NodeType* orig_head = *root;
    // first step, get snapshot of head
    //(can change during runtime, use only copy)

    // will keep the previous node
    // which will be the connection point

    ...
}
```

Next, it uses the node's `traversalDone` and `nextChild` methods to find the connection point and record the path to it.

```
...
NodeType* curr = orig_head;

// search for node with key
for (; curr && !curr->traversalDone(key);) {

    auto next_child = curr->nextChild(key);

    // search for node with key, adding path to stack and keeping the prev
    result.path.push(curr, next_child);

    curr = curr->getChild(next_child);
};
...
```

Finally, it uses the path to populate the information required by `ConnPointData`.

```
    int prev_next_child_index = INSERT_POSITIONS::AT_ROOT;

    NodeType* prev_s = nullptr;

    if (!result.path.Empty()) {
        auto stack_top =  result.path.pop();
        prev_s = stack_top.node;
        prev_next_child_index = stack_top.next_child;
    }


    // was a node found
    result.found_ = curr && curr->hasKey(key);

    // set prev as the connection point
    // will be null if operation happens at head
    result.connection_point_ = prev_s;

    // the child to which the pointer to change corresponds
    // should be AT_ROOT if there's no connection_point
    result.con_ptr.child_index = prev_next_child_index;
```

```
    // the original value of the pointer to change
    result.con_ptr.snapshot = result.connection_point_ ? curr : orig_head;

    return result;
}
```

3.3.2.1 Lookup Functions

Without constructing a `ConnPointData` object or saving the path, the library also offers:

- `NodeType* find<NodeType>(NodeType* root, int desired_key)`, which searches for a given key.
- `bool node_exists(NodeType* target_node, NodeType* root)`, which tests whether a given node belongs to the tree.

3.3.3 PathTracker

`PathTracker` allows searching in any tree structure. It is parameterized by the node type and effectively acts as a state machine representing a tree traversal.

3.3.3.1 Initialization

It is initialized with the address of the root pointer. This indirection is needed to build `ConnPointData`.

3.3.3.2 Structure

The object tracks four things:

1. The root.
2. The node at which it currently points (its *current position*).
3. The depth of the current position in the tree.
4. The path from the root to the current position.

3.3.3.3 Current Position and Connection Point

The current position designates where the copy tree will be attached. From the saved path, the parent of that position can be identified as the connection point and the current position as the child to be changed.

3.3.3.4 Movement

At initialization, the current position is the root of the original tree. Movement is supported in two directions:

1. Toward the children with `moveToChild(int pos)`, which moves the current position to child number `pos`.

```
NodeType* moveToChild(int pos) {
    ++at_level_;

    path_.push(current_pos_, pos);
    current_pos_ = current_pos_->getChild(pos);

    return current_pos_;
}
```

   This movement increases the depth by one, records the path, and returns the new position.

2. Toward the root with `moveUp(int n_levels = 1)`, whose optional parameter specifies how many levels to ascend.

```
void moveUp(int n_levels = 1) {
    at_level_ = at_level_ >= n_levels ? at_level_ - n_levels: -1;

    for (int i = 0; at_level_ >= 0 && i < n_levels - 1; i++) {
        path_.pop();
    }

    if (!path_.Empty()) {
        current_pos_ =  path_.pop().node;
    }
}
```

   Movement decreases the recorded depth by `n_levels`, with the terminal level (at the root) represented by -1.

Defining a Connection Point The ultimate purpose of `PathTracker` is to define the connection point through `connectHere()`. When called, it removes the predecessor of the current position from the path, uses that node as the connection point, and returns the corresponding `ConnPointData` object, built as in `find_conn_point`.

```
ConnPointData<NodeType> connectHere() {
    // create a ConnPointData object
    ConnPointData<NodeType> conn_point_snapshot;

    // conn point is at root level
    bool at_root = at_level_ == -1;

    // get prev node from path
    NodeAndNextPointer<NodeType> conn_point_and_next_child = path_.pop();

    // set it
    conn_point_snapshot.connection_point_ = at_root ? nullptr: conn_point_and

    // record path in ConnPointData
    path_.copy_to(conn_point_snapshot.path);


    // fill ConnPointData
    conn_point_snapshot.root_of_structure = root_;

    conn_point_snapshot.con_ptr.child_index = at_root?
            INSERT_POSITIONS::AT_ROOT :
                conn_point_and_next_child.next_child;

    conn_point_snapshot.con_ptr.snapshot = current_pos_;

    // restore path
    path_.push(
        conn_point_and_next_child.node,
        conn_point_and_next_child.next_child
    );

    // return ConnPointData
    return conn_point_snapshot;
}
```

3.3.4 PathTracker Thread Safety

`PathTracker` is designed for use outside transactions while searching for a connection point. Such a search *guarantees that it supplies correct information to the library to execute a thread-safe operation*, but **does not guarantee that all nodes encountered during the traversal jointly form a consistent snapshot of the tree**.

For example, a user might be tempted to employ it for a more complex traversal such as an in-order traversal. `PathTracker` would not guarantee that the resulting order represents a valid tree state; it guarantees only that *the path from the root to each visited node was valid at the moment that node was visited.*

3.4 The Copy Tree

3.4.1 Developing the Idea

The copy tree was inspired by the concept of monads in functional programming. We want to describe a computation with side effects—changing an arbitrary tree—using code that looks sequential but has no side effects while being executed, so that many threads may run it in parallel. At the end of the operation, of course, the changes should become visible in the original tree.

3.4.1.1 Assumption

Suppose an operation modifies a portion of the tree: a subtree attached to some parent, or the root itself. Nodes in that subtree have children that are also part of the modified subtree, as well as children belonging to the original tree that do not change. Suppose we copy only this subset, producing a new tree to which we apply the required changes, while remembering which original nodes were copied.

3.4.1.2 Atomic Connection

As established for RCU-HTM, if we atomically verify that:

1. The original children of the modified nodes have not changed during the operation.
2. The subtree of original nodes is still connected to the original tree.

then atomically swapping the original subtree with its modified copy preserves tree consistency.

3.4.1.3 The Copy

We therefore need a representation of a copy of a subtree in which changes occur. It must retain:

1. The original child pointers as they existed when copied, to verify condition (1).
2. References to the original nodes, also for condition (1).
3. References to the new copied nodes.
4. A reference to the original subtree root, to verify condition (2).
5. Changes to links and data values in the copied nodes.

Since the structure describes changes to a tree, a natural choice is to make it a tree itself. Each thread independently constructs and maintains such a structure in order to modify the original tree.

3.5 Safe Node

We thus arrive at the nodes of the copy tree: *safe nodes*. These are intended to hold the information described above while representing modifications to any tree structure. They are constructed based on nodes from the original tree.
A safe node holds:

1. A reference to a `ConnPoint` object (at the root).
2. A reference to the original node from which it was constructed.
3. A reference to the copied node, which may contain modifications.
4. References to other safe nodes as its children.
5. A bitmap indicating which original children have been accessed during the current operation and have associated safe nodes.
6. The original nodes' child pointers at the time of copying.
7. A type indicator recording whether the original node belonged to the original tree, was newly created for insertion, or otherwise does not require validation.

3.5.1 Definition

```
template <class NodeType>
class SafeNode
{
    enum SAFENODE_TYPE {
        ORIG_TREE_NODE,
        ORIG_TREE_NO_VALIDATION,
        NEW_NODE
    };

    private:
        ConnPoint<NodeType>& conn_point_;
        NodeType* const original_;
        NodeType* copy_;
        std::array<SafeNode<NodeType>*,NodeType::maxChildren()> children_;
        std::array<bool,NodeType::maxChildren()> modified_;
        std::array<NodeType*, NodeType::maxChildren()>
        children_pointers_snapshot_;

        bool deleted_; // for deletion

        SAFENODE_TYPE node_type_;
        ...
}
```

The properties of the safe node:

1. Give users the impression that they are modifying the original tree, while actually constructing a copy tree to be attached later.

2. Allow `getChild` to *expand the copy tree*: the method automatically wraps the corresponding original child in a new safe node and connects their internal copies.

3. Provide a *safe reference* through `rwRef`, returning a pointer to the copy (which may be modified without affecting the original tree) and simulating sequential mutation of node values.

4. Simulate sequential changes to child links through `setChild`, which deliberately accepts only safe nodes and also updates connections among copies.

5. Make the copy tree **persistent** within the operation: expansions performed by `getChild` automatically, or by `setChild` explicitly, are stored as children of safe nodes. Later accesses do not recreate those nodes and copies.

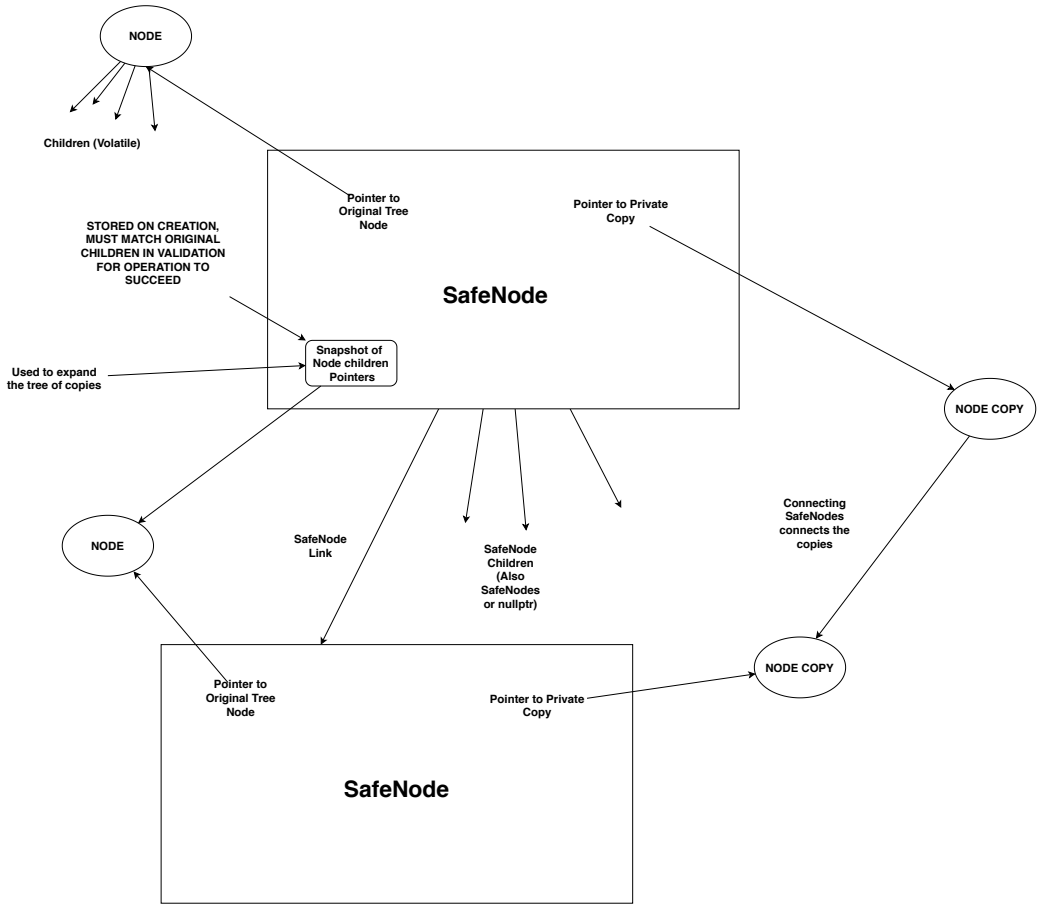


Figure 3.1: Safe Node Implementation

3.5.2 The getChild Method

The `getChild` method illustrates the mechanism particularly well, since it effectively constructs the copy tree.
It takes as its parameter the index of the safe node's child to access.

```
//getChild: returns either an already linked SafeNode
//or creates a new SafeNode when accessing an unsafe node
SafeNode<NodeType>* getChild(const int child_pos) {
    /* process:
    A SafeNode without set children will have nullptr in
    its array. When getting the child of a SafeNode:
    if:
        1. has been set) return it
        2. has not been set) create SafeNode, assign to children and return it

    */
    ...
}
```

First, the method checks that the index is valid and that the safe node's type permits safe access to its children. If validation has been disabled, accessing its children is prohibited because that would not be thread-safe.

Next, if the node has not been copied and belongs to the original tree, a copy is created (before copying, the copy pointer points to the original node). Why copy the parent when we only want to access a child? Because safe nodes manage the connections of their copies, not the originals. To keep the copy tree consistent, every safe node that has other safe nodes as children needs to be copied.

Calling getChild indicates that we intend to modify the nodes being accessed. If all we need is to inspect a safe node's child pointers, we should use `peekChild`, which creates no copies.

```
 // no oob errors
assert(child_pos >= 0 && child_pos < NodeType::maxChildren());
assert(node_type_ != ORIG_TREE_NO_VALIDATION && "Validation Error: Tried to access a

// force parent copy
// getting a child
// explicitly
```

```
// means that you
// seek to modify it
if (copy_ == original_ && node_type_ == ORIG_TREE_NODE) {
    make_copy();
}
...
```

The `modified_` bitmap is then checked: if an earlier access already created a safe node, that stored node is returned rather than creating another.

```
...

//child has already been copied
if (modified_[child_pos]) {
    return children_[child_pos];
}

...
```

On the first access, a new safe node is constructed around the corresponding child of the original node, added as a child of the calling safe node, and returned.

The calling node's type determines the types of its children. If the parent safe node represents a node from the original tree (`ORIG_TREE_NODE`), the safe nodes produced by `getChild` have the same type. If it represents a newly created node, all its children are also considered newly created.

```
// child has been modified
modified_[child_pos] = true;

SafeNode<NodeType>* new_safe = nullptr;

// if node has been copied read from copy
// else read from original child pointers backup
const auto original_child = copy_->getChild(child_pos);

 //child not copied yet,
//copy and use the copy from now on
if (original_child) {

    new_safe = node_type_ == ORIG_TREE_NODE ?
    conn_point_.wrap_safe(original_child):
        conn_point_.create_safe(original_child);

    children_[child_pos] = new_safe;
    copy_->setChild(child_pos, new_safe->rwRef());
}

return new_safe;
```

3.5.3 The setChild Method

`setChild` preserves the representation already described while allowing any safe node to be attached as a child of the calling safe node. It can attach a newly created node when combined with `ConnPoint`'s `createSafe` method.

3.5.4 Relationship with ConnPoint

Every copy tree is defined by one `ConnPoint` and its safe nodes. Thus each safe node uniquely references the `ConnPoint` that created it, allowing that connection-point object to access the constituent safe nodes. The `ConnPoint` also maintains a validation set containing references to all safe nodes whose underlying original nodes require validation.

To enforce this relationship, safe nodes cannot be constructed directly (their constructors are private). They must be created either through a `ConnPoint` or through another safe node, inheriting its connection-point reference.

3.5.5 Safe-Node Types

The safe-node types are:

1. A node from the original tree, indicating that the wrapped node was already in that tree.
2. A newly created node, which is not added to the validation set.
3. A node exempt from validation, likewise excluded from the validation set.

These types anticipate a future memory-management mechanism, in which the library must know which nodes may safely be freed on transaction failure and which must be retained.

3.6 Connection Point (ConnPoint)

3.6.1 Connection-Point Structure

The `ConnPoint` object brings together all components of the library. For concision, its internal references and variables are reproduced in code form, since they collectively illustrate the objects and references that it manages.

```
// returns if copy connection was successful
    bool& connect_success_;

    // preallocated pool of SafeNodes
    thread_local static PreAllocVec<SafeNode<NodeType>*, 500> validation_set_;

    // the connection point is the node whose child pointer will
    // be modified to connect the tree of copies
    NodeType* connection_point_;

    // if null, the insertion should be done at
    // the root of the data structure

    // the point of insertion of the new tree
    NodeType** connection_pointer_;

    // the root of the data structure to modify
    NodeType** root_;

    // original connection pointer
    // should not have changed during transaction
    NodeType* conn_pointer_snapshot_;


    // if a node is popped from path,
    // to be set as the new connection point,
    // the old head will be its child at index
    // child_to_exchange_
    int child_to_exchange_;

    // path to connection point
    Stack& path_to_conn_point_;
```

```
// tree of copies was successfully connected
bool copy_connected_;


// The global lock
TSX::SpinLock& _lock;

// tsx stats
TSX::TSXStats &stats_;


// the root node of the tree of copies
SafeNode<NodeType>* head_;

// determines if any modification
// was done to the tree
bool tree_was_modified_;

// won't run the operation
// using transaction if lock is already locked
bool already_locked_;

// transactional retries before
// acquiring fallback lock
int& trans_retries_;

// set to true if operation
// throws a validation error
bool validation_aborted_;
```

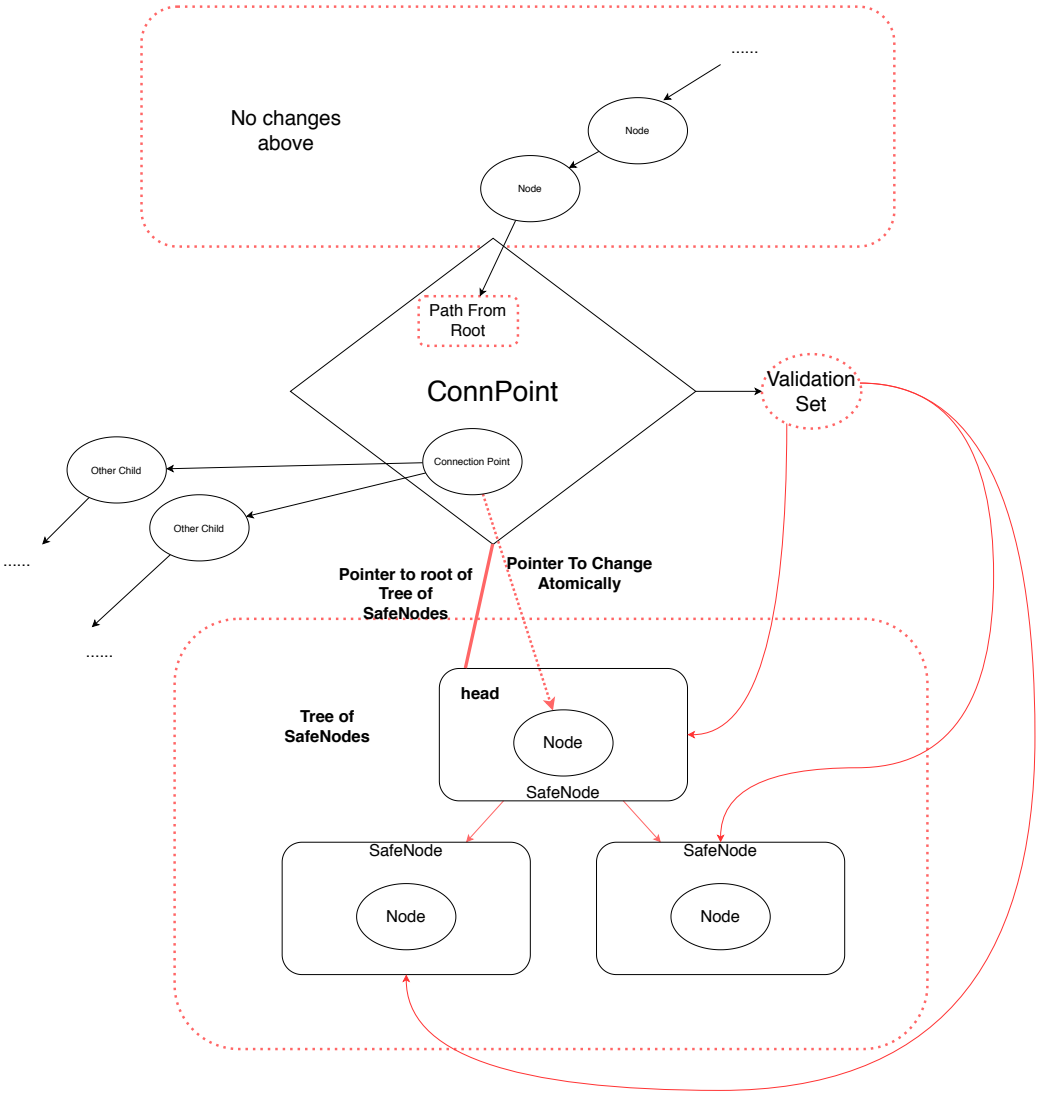


Figure 3.2: ConnPoint Implementation

3.6.2 Operations

`ConnPoint` is the library's most important object because it performs nearly all the HTM-related work while hiding it from the user.

It is constructed from the `ConnPointData` produced by connection-point search.

Its duties include:

1. Acting as the root owner of the copy tree and creating its first safe node.
2. Recording the original-tree connection point and the child at which the copy tree will be attached.
3. Connecting the copy tree to the original tree if the transaction succeeds.
4. Constructing safe nodes around original or newly created nodes.
5. Managing validation with access to all safe nodes participating in the operation.
6. Moving the connection point toward the root when needed.

3.6.3 The Copy-Tree Root

Internally, `ConnPoint` maintains a distinct tree of safe nodes, represented by a pointer to its root. This tree is empty at object construction, so the root pointer is `nullptr`.

Calling `getRoot` behaves analogously to a safe node's `getChild`: it wraps the child at the attachment position in a safe node and starts the copy tree. Likewise, `setRoot` sets the copy-tree root to the safe node chosen by the user.

3.6.4 Connecting to the Original Tree

Given the connection point, the child index at which the copy tree should be installed (or the fact that the original root itself should be replaced), we can explain what attaching the copy tree means.

The copy tree consists of safe nodes, whereas the original tree consists of user-defined nodes. When the connection is made, the connection point (or root pointer) is updated to point to the user-node copy held internally by the copy tree's root safe node, or to `nullptr`.

Safe nodes ensure that the copies they hold form a correctly linked tree reflecting the user's changes. Consequently, attaching the root of that tree as a child of the connection point is sufficient.

The connection is made inside an HTM transaction after the original tree's consistency has been validated, and is therefore atomic.

3.6.5 Constructing Safe Nodes

`ConnPoint` is the only object responsible for constructing safe nodes. Its `wrap_safe`, `create_safe` and `wrap_no_validate` methods invoke the safe-node constructor, passing a self-reference (`this`), ensuring the one-to-one relationship between each safe node and its `ConnPoint`.

```
SafeNode<NodeType>* wrap_safe(NodeType* some_node) {
    if (!some_node) {
        return nullptr;
    }

    return new SafeNode<NodeType>(*this, some_node);
}
```

3.6.6 Validation

Overview Copy-tree validation ensures that the copy tree can be attached without compromising the consistency of the original tree. It runs inside an HTM transaction (so validation itself is atomic) and consists of:

1. Verifying that the connection point remains connected to the original tree.

2. Verifying that the pointer to the child to be replaced has not changed, nor have any child pointers belonging to copy-tree nodes (or, more generally, original nodes wrapped in safe nodes during the operation).

Atomicity Since all checks occur within an HTM transaction, any conflicting change by another thread to a pointer read by validation causes the transaction to abort. Thus the checks are atomic.

Connection-point membership This is checked differently depending on whether the structure is a search tree or a general tree.

- For search trees, the generated `find_target_node` routine traverses the original tree from its root to locate the connection point, using each node's `next_child(NodeType* target_node)` method to choose the direction. This is preferable because it is enough for the connection point to remain somewhere in the tree, even if the path leading to it has changed.

- For general trees, the library validates the original traversal path to the connection point, retained in the `ConnPoint` object.

Child-pointer checks `ConnPoint` saves the pointer value of the connection point's child to detect changes. For other nodes of the copy tree, it uses the validation set.

Validation set At construction, each safe node records a snapshot of the child pointers of the node it wraps. With both this snapshot and a reference to the original node, it can test whether any child pointers have changed.

The validation set is an array of references to all safe nodes requiring validation.

When created, every safe node of type `NODE_FROM_ORIG_TREE` inserts itself into its owning `ConnPoint`'s validation set.

```
bool validate_copy() {
   if (!connection_point_) {
       // insertion at root

       // has root copy changed ?
       if (conn_pointer_snapshot_ != *root_) {
           TSX::TSXGuard::abort<VALIDATION_FAILED>();
           return false;
       }
   } else {
       // has the connection pointer to be modified
       // changed?
       if (conn_pointer_snapshot_ != connection_point_->getChild(child_to_exchange_
           TSX::TSXGuard::abort<VALIDATION_FAILED>();
           return false;
       }

       // is the root of the tree of copies still connected?
       if (!connPointConnected()) {
           TSX::TSXGuard::abort<VALIDATION_FAILED>();
           return false;
       }
   }

   return validate_children(...);

   }
```

```
bool validate_children() {
    // have the children pointers of the original nodes changed?
    for (auto i = 0; i < validation_set_.size(); i++) {
            auto safe_node = validation_set_.get(i);
            // skip for new nodes
            if (safe_node->node_type_ == SafeNode<NodeType>::ORIG_TREE_NODE)
                // the original node and its current children
                const auto original_node = safe_node->original_;

                // the saved children which shouldn't have changed
                const auto &saved_children = safe_node->children_pointers_sna

                for (int i = 0; i < NodeType::maxChildren(); i++) {
                    if (saved_children[i] != original_node->getChild(i)) {
                        TSX::TSXGuard::abort<VALIDATION_FAILED>();
                        return false;
            }
        }
    }
}
```

Validation failure Validation can fail in the following cases:

1. One of the checks fails. The library issues an `abort`, restarting the transaction or the entire operation.
2. A concurrent access by another thread aborts the transaction.
3. The transaction aborts for an unrelated reason, such as a page fault.

3.6.7 Moving the Connection Point

We have already discussed several times why it is useful to move the connection point toward the root. Here we explain only the mechanism.

Given a connection point and the saved path to it, moving the copy tree's connection point to its parent requires making the former connection point the copy tree's root.

Let us examine the method implementing this behavior:

First, the preceding node is removed from the saved path and prepared to become the new connection point. The path also contains the child index corresponding to the former connection point.

```
SafeNode<NodeType>* pop_path() {

    // is at root
    // return nullptr
    if (!connection_point_) {
        return nullptr;
    }


    NodeType* new_conn_point = nullptr;
    int new_conn_point_next_index = -1;

    if (!path_to_conn_point_.Empty()) {

        // pop node
        auto parent_of_conn_point = path_to_conn_point_.pop();

        new_conn_point = parent_of_conn_point.node;

        new_conn_point_next_index = parent_of_conn_point.next_child;

    }
...
```

Next, the address of the old connection point (connpoint_) is saved as the child-pointer value to be replaced atomically, and its index is also saved for validation. The new connection point is then set.

```
// keep old conn pointer snapshot to force its validation
auto old_conn_pointer_snapshot_ = conn_pointer_snapshot_;

// for the new connection point
// the pointer to change
// should point to the previous
// connection point
conn_pointer_snapshot_ = connection_point_;

// calculate which child should be changed to insert the tree of copies
// by using the previous connection point's key
const int next_child_index = new_conn_point_next_index;


// change connection point to the new conn point
connection_point_ = new_conn_point;

// also update connection pointer
// if not at root
if (connection_point_) {
    connection_pointer_ = connection_point_->getChildPointer(next_child_index
}
```

Finally, the copy tree is extended toward the original root by wrapping the old connection point, making it the copy tree's root, and attaching the existing copy tree as a child of it. An interesting detail is the check that the new copy-tree root still has the former copy-tree root as its child, which is necessary to preserve consistency. The library offers two ways of enforcing this condition through a compilation flag. With TM_EARLY_ABORT enabled, it checks the condition immediately, outside a transaction, and restarts the whole operation if it fails. Otherwise, the condition is encoded in the stored child pointers of the new root, to be tested during validation.

```
    ....

    // using the previous connection point
    // make a safeNode to turn to new head
    SafeNode<NodeType>* newHead = wrap_safe(conn_pointer_snapshot_);


    // and connect it
    newHead->setChild(child_to_exchange_, head_);


    // the old connection point should necessarily continue
    // pointing to it's old value
    #ifdef TM_EARLY_ABORT
        if (newHead->original->getChild(child_to_exchange_)
                != old_conn_pointer_snapshot_) {
            validation_abort();
            throw ValidationAbortException();
        }
    #endif

    newHead->children_pointers_snapshot_[child_to_exchange_]
                                    = old_conn_pointer_snapshot_;


    // now head can be updated
    head_ = newHead;
    // as well which child to update for the next pop
    child_to_exchange_ = next_child_index;

    return head_;
}
```

3.6.8 Completing an Operation

When the transaction block's code finishes, the `ConnPoint` destructor is called. It invokes the method that attaches the copy tree.

```
bool connect_atomically() noexcept {

    if (validation_aborted_) {
        return false;
    }

    unsigned char err_status = 0;

    TSX::TSXTransOnlyGuard guard(
        trans_retries_,
        _lock,
        err_status,
        stats_,
        already_locked_,
        TSX::STUBBORN
    );

    if (err_status != VALIDATION_FAILED && validate_copy()) {
        connect_copy();
        return true;
    }

    return false;
}
```

As the method shows, an atomic section is created in which validation and connection are attempted. If validation finds a consistency error, the constructed copy tree is discarded and the operation restarts by **returning false**. If the attempt fails for some other reason, validation may be retried in case it succeeds on a subsequent attempt.

Ultimately, the copy tree is installed and the method **returns true**.

3.7 Pools

The library must create many safe-node objects (an overhead of the abstraction), as well as original-tree nodes (for example, during insertion). These are ordinarily allocated on the heap, requiring memory-management functions that significantly reduce performance. Such functions are not only expensive but also use internal synchronization mechanisms that may scale poorly, sharply reducing the scalability of concurrent data structures. Specialized multithreaded memory allocators help, but a more efficient solution is available.

For safe nodes in particular, we know:

- Tree operations usually modify or add relatively few nodes.
- Safe nodes are retired whether a transaction succeeds or fails.

Thus, if we preallocate memory for a small number of safe nodes separately for each thread, we can repeatedly reuse that memory and virtually eliminate allocations during library operations.

For original nodes, we use the same technique to separate measured library performance from the influence of the chosen allocator: a large memory area is allocated during library initialization, and nodes are constructed from that area.

Implementation The implementation uses C++ thread-local variables, which allow each thread to have its own static variables managed through common code.

For safe nodes, we implemented a `memory_pool` class providing access to a given number of preallocated safe-node objects. Each `ConnPoint` class has access to a thread-local `memory_pool` (one per thread), enabling node reuse. Users do not interact directly with this mechanism.

For original nodes, the `memory_pool_tracked` object works similarly but supports deallocation of allocated memory. `ConnPoint` thus exposes its own allocator, backed by a thread-local `memory_pool_tracked` (one per thread), and a `create_new_node` method accepting the same constructor arguments as `new`. Before use, `init_node_pool` must be called in the original tree's constructor; `reset_node_pool` should be called in its destructor to reset the pools.

Limitations The current implementation has no memory-reclamation mechanism, so memory occupied by original nodes is never released except when `reset_node_pool` frees all nodes associated with a given tree.

Furthermore, since this mechanism was designed primarily for benchmarks, the pools currently support only a single tree per executable, because the library objects' pools are static variables.

3.8 Early Operation Abortion

The library supports an optional mode that uses C++ exceptions to abort an operation early, avoiding unnecessary initiation of an HTM transaction that might conflict with other transactions.

In this mode, while copies of nodes wrapped by safe nodes are being constructed, the library checks that the original node's children have not changed since the safe node was created, similar to validation inside a transaction.

```
for (int i = 0; i < NodeType::maxChildren(); i++) {
    #ifdef TM_EARLY_ABORT_ALL
        if (children_pointers_snapshot_[i] != original_->getChild(i)) {
            conn_point_.validation_abort();
            throw ValidationAbortException();
        }
    #endif

    copy_->setChild(i,children_pointers_snapshot_[i]);
}
```

The exception is handled transparently by the transaction block (here the closing `TM_SAFE_OPERATION_END` macro is required because it contains the handler), effectively restarting the operation.

Chapter 4

Evaluation

4.1 Code Size and Simplicity

The first metric we evaluate is code size. We use the number of lines of code—admittedly an imperfect measure—to compare an AVL tree implemented using the library with a manual RCU-HTM implementation by my thesis advisor Dimitris Siakavaras.

Since these implementations are written in C++ and C, respectively, we compare only the insertion-method implementations, including auxiliary methods implementing RCU-HTM functionality. We also compare how easy the code is to understand.

4.1.0.1 Manual RCU-HTM

We begin with a manually implemented node insertion operation, which is not easy to understand:

```
1  int rbt_insert(void *rbt, void *thread_data, int key, void *value)
2  {
3      int ret = 0;
4      ret = _avl_insert_helper(rbt, key, value, thread_data);
5      return ret;
6  }
7
8  static int _avl_insert_helper(avl_t *avl, int key, void *value, tdata_t *tda
9  {
10     avl_node_t *nodes_to_free[MAX_htEIGhtT], *nodes_alloced[MAX_htEIGhtT];
11     int ntf_top, nalloced_top;
12     avl_node_t *node_stack[MAX_htEIGhtT];
```

```
13      int stack_top;
14      tm_begin_ret_t status;
15      int retries = -1;
16      int i;
17      avl_node_t *tree_copy_root, *connection_point;
18      int connection_point_stack_index;
19
20  try_from_scratch:
21      ht_reset(tdata->ht);
22
23
24      if (++retries >= TX_NUM_RETRIES) {
25          tdata->lacqs++;
26          pthread_spin_lock(&avl->avl_lock);
27          _traverse_with_stack(avl, key, node_stack, &stack_top);
28          if (stack_top >= 0 && node_stack[stack_top]->key == key) {
29              pthread_spin_unlock(&avl->avl_lock);
30              return 0;
31          }
32          connection_point_stack_index = -1;
33          connection_point = _insert_and_rebalance_with_copy(key, value,
34                                      node_stack, stack_top, tdata, &tree_copy_root,
35                                      &connection_point_stack_index,
36                                      nodes_to_free, &ntf_top,
37                                      nodes_alloced, &nalloced_top);
38          tdata->replaced_nodes += ntf_top + 1;
39          if (!connection_point) {
40              avl->root = tree_copy_root;
41          } else {
42          if (key <= connection_point->key)
43                  connection_point->left = tree_copy_root;
44          else
45                  connection_point->right = tree_copy_root;
46          }
47          pthread_spin_unlock(&avl->avl_lock);
48
49      /* Asynchronized traversal. If key is there we can safely return. */
50      _traverse_with_stack(avl, key, node_stack, &stack_top);
51      if (stack_top >= 0 && node_stack[stack_top]->key == key) {
52          return 0;
53      }
```

```
54
55      // For now let's ignore empty tree case and case with only one node in
56      assert(stack_top >= 2);
57
58      connection_point_stack_index = -1;
59      connection_point = _insert_and_rebalance_with_copy(key, value,
60                                      node_stack, stack_top, tdata, &tree_copy_roo
61                                      &connection_point_stack_index,
62                                      nodes_to_free, &ntf_top,
63                                      nodes_alloced, &nalloced_top);
64      tdata->replaced_nodes += ntf_top + 1;
65
66      int validation_retries = -1;
67  validate_and_connect_copy:
68
69      if (++validation_retries >= TX_NUM_RETRIES) {
70      goto try_from_scratch;
71      }
72
73      /* Transactional verification. */
74      while (avl->avl_lock != LOCK_FREE)
75          ;
76
77      tdata->tx_starts++;
78      status = TX_BEGIN(0);
79      if (status == TM_BEGIN_SUCCESS) {
80          if (avl->avl_lock != LOCK_FREE)
81              TX_ABORT(ABORT_GL_TAKEN);
82
83          // Validate copy
84          if (key < node_stack[stack_top]->key && node_stack[stack_top]->left !
85              TX_ABORT(ABORT_VALIDATION_FAILURE);
86          if (key > node_stack[stack_top]->key && node_stack[stack_top]->right
87              TX_ABORT(ABORT_VALIDATION_FAILURE);
88          if (avl->root != node_stack[0])
89              TX_ABORT(ABORT_VALIDATION_FAILURE);
90
91          if (connection_point_stack_index <= 0) {
92              for (i=0; i < stack_top; i++) {
93                  if (key <= node_stack[i]->key) {
94                          if (node_stack[i]->left != node_stack[i+1])
```

```
95                          TX_ABORT(ABORT_VALIDATION_FAILURE);
96                  } else {
97                      if (node_stack[i]->right!= node_stack[i+1])
98                          TX_ABORT(ABORT_VALIDATION_FAILURE);
99                  }
100             }
101         } else {
102             avl_node_t *curr = avl->root;
103             while (curr && curr != connection_point)
104                 curr = (key <= curr->key) ? curr->left : curr->right;
105             if (curr != connection_point)
106                 TX_ABORT(ABORT_VALIDATION_FAILURE);
107             for (i=connection_point_stack_index; i < stack_top; i++) {
108                 if (key <= node_stack[i]->key) {
109                     if (node_stack[i]->left != node_stack[i+1])
110                         TX_ABORT(ABORT_VALIDATION_FAILURE);
111                 } else {
112                     if (node_stack[i]->right!= node_stack[i+1])
113                         TX_ABORT(ABORT_VALIDATION_FAILURE);
114                 }
115             }
116         }
117
118         int j;
119         for (i=0; i < htT_LEN; i++) {
120             for (j=0; j < tdata->ht->bucket_next_index[i]; j+=2) {
121                 avl_node_t **np = tdata->ht->entries[i][j];
122                 avl_node_t  *n  = tdata->ht->entries[i][j+1];
123                 if (*np != n)
124                     TX_ABORT(ABORT_VALIDATION_FAILURE);
125             }
126         }
127
128
129         // Now let's 'commit' the tree copy onto the original tree.
130         if (!connection_point) {
131             avl->root = tree_copy_root;
132         } else {
133             if (key <= connection_point->key)
134                 connection_point->left = tree_copy_root;
135             else
```

```
136                 connection_point->right = tree_copy_root;
137         }
138
139         TX_END(0);
140     } else {
141         tdata->tx_aborts++;
142         if (ABORT_IS_EXPLICIT(status) &&
143             ABORT_CODE(status) == ABORT_VALIDATION_FAILURE) {
144             tdata->tx_aborts_explicit_validation++;
145             goto try_from_scratch;
146         } else {
147             goto validate_and_connect_copy;
148         }
149 }
150
151
152 static inline void _traverse_with_stack(avl_t *avl, int key,
153                                         avl_node_t *node_stack[MAX_htEIGhtT],
154                                         int *stack_top)
155 {
156     avl_node_t *parent, *leaf;
157
158     parent = NULL;
159     leaf = avl->root;
160     *stack_top = -1;
161
162     while (leaf) {
163         node_stack[++(*stack_top)] = leaf;
164
165         int leaf_key = leaf->key;
166         if (leaf_key == key)
167             return;
168
169         parent = leaf;
170         leaf = (key < leaf_key) ? leaf->left : leaf->right;
171     }
172 }
173
174 static avl_node_t *_insert_and_rebalance_with_copy(int key, void *value,
175         avl_node_t *node_stack[MAX_htEIGhtT], int stack_top, tdata_t *tdata,
176         avl_node_t **tree_copy_root_ret, int *connection_point_stack_index,
```

```
177         avl_node_t *nodes_to_free[MAX_htEIGhtT], int *ntf_top,
178         avl_node_t *nodes_alloced[MAX_htEIGhtT], int *nalloced_top)
179 {
180     avl_node_t *tree_copy_root, *connection_point;
181 
182     *ntf_top = -1;
183     *nalloced_top = -1;
184 
185     //> Start the tree copying with the new node.
186     tree_copy_root = avl_node_new(key, value);
187     nodes_alloced[++(*nalloced_top)] = tree_copy_root;
188     *connection_point_stack_index = stack_top;
189     connection_point = stack_top >= 0 ? node_stack[stack_top--] : NULL;
190 
191     //> Empty tree case
192     if (stack_top < 0) {
193         *tree_copy_root_ret = tree_copy_root;
194         return connection_point;
195     }
196 
197     while (stack_top >= -1) {
198         //> If we've reached and passed root return.
199         if (!connection_point)
200             break;
201 
202         //> If no height change occurs we can break.
203         if (tree_copy_root->height + 1 <= connection_point->height)
204             break;
205 
206         //> Copy the current node and link it to the local copy.
207         avl_node_t *curr_cp = avl_node_new_copy(connection_point, tdata);
208         nodes_to_free[++(*ntf_top)] = connection_point;
209         nodes_alloced[++(*nalloced_top)] = curr_cp;
210         ht_insert(tdata->ht, &connection_point->left, curr_cp->left);
211         ht_insert(tdata->ht, &connection_point->right, curr_cp->right);
212 
213         curr_cp->height = tree_copy_root->height + 1;
214         if (key < curr_cp->key) curr_cp->left = tree_copy_root;
215         else                    curr_cp->right = tree_copy_root;
216         tree_copy_root = curr_cp;
217 
```

```
218             // Move one level up
219             *connection_point_stack_index = stack_top;
220             connection_point = stack_top >= 0 ? node_stack[stack_top--] : NULL;
221 
222             // Get current node's balance
223             avl_node_t *sibling;
224             int curr_balance;
225             if (key < curr_cp->key) {
226                 sibling = curr_cp->right;
227                 curr_balance = node_height(curr_cp->left) - node_height(sibling);
228             } else {
229                 sibling = curr_cp->left;
230                 curr_balance = node_height(sibling) - node_height(curr_cp->right)
231             }
232 
233             if (curr_balance == 2) {
234                 int balance2 = node_balance(tree_copy_root->left);
235 
236                 if (balance2 == 1) { // LEFT-LEFT case
237                     tree_copy_root = rotate_right(tree_copy_root);
238                 } else if (balance2 == -1) { // LEFT-RIGhtT case
239                     tree_copy_root->left = rotate_left(tree_copy_root->left);
240                     tree_copy_root = rotate_right(tree_copy_root);
241                 } else {
242                     assert(0);
243                 }
244 
245                 break;
246             } else if (curr_balance == -2) {
247                 int balance2 = node_balance(tree_copy_root->right);
248 
249                 if (balance2 == -1) { // RIGhtT-RIGhtT case
250                     tree_copy_root = rotate_left(tree_copy_root);
251                 } else if (balance2 == 1) { // RIGhtT-LEFT case
252                     tree_copy_root->right = rotate_right(tree_copy_root->right);
253                     tree_copy_root = rotate_left(tree_copy_root);
254                 } else {
255                     assert(0);
256                 }
257 
258                 break;
```

```
259             }
260         }
261 
262         *tree_copy_root_ret = tree_copy_root;
263         return connection_point;
264     }
```

This implementation consists of approximately 264 lines of fairly obscure code, diverging considerably from a sequential implementation. It includes:

1. Synchronization hazards arising from the implementation of transactions.

2. Complicated copying logic.

3. The need to pass many parameters between functions, several related solely to synchronization rather than the data structure's logic.

4. Numerous boundary checks that could easily be affected by human error.

It also makes debugging extremely difficult because of the number of variables and offers limited opportunities to reuse the code for other structures. Finally, it bears little resemblance to a sequential implementation, because it must implement the entire logic of our library separately for each operation using cumbersome primitives.

4.1.0.2 Library

Now consider the version implemented using the library:

```
 bool insert(const int k, ValueType val, int t_id) {
        return insert_impl(k,val, t_id);
}

// the insert operation
bool insert_impl(const int k, ValueType val, int t_id) {

    int retries = trans_retries;

    TM_SAFE_OPERATION_START {

        TSX::Transaction t(retries,_lock, stats[t_id]);

        /* FIND PhtASE */

        auto conn_point_snapshot = find_conn_point<TreeNode>(k,&root);


        if (conn_point_snapshot.found()) {
            return false;
        }


        /* FIND PhtASE END */

        ConnPoint<TreeNode> conn(conn_point_snapshot, t);

        /* INSERT */

        auto node_to_be_inserted = conn.create_safe(
        ConnPoint<TreeNode>::create_new_node(k,val,nullptr,nullptr));


        // insert it
        conn.setRoot(node_to_be_inserted);

        // identical to bst up to this point

```

```
39
40          // if not inserting at root
41          if (conn_point_snapshot.connection_point()) {
42
43              bool rotation_happened = false;
44
45              /* REBALANCE */
46
47              // go up path and rebalance
48              for (SafeNode<TreeNode>* n = conn.pop_path();
49                                          n != nullptr; n = conn.pop_path()) {
50
51                  auto n_data = n->rwRef();
52                  int height_old = n_data->height;
53
54                  n = rebalance_ins(n, k , rotation_happened);
55                  // apply rebalancing to all
56                  // required nodes
57                  // rotation returns the new root to place
58                  // below the connection point
59
60                  conn.setRoot(n);
61                  // change the root of the
62                  // tree of copies to make the change visible
63
64                  if (height_old == n_data->height && !rotation_happened) {
65                      break;
66                  }
67
68              }
69          }
70      } TM_SAFE_OPERATION_END
71
72      return true;
73  }
74
75     // apply rebalancing for an insertion operation
76          SafeNode<TreeNode>* rebalance_ins(SafeNode<TreeNode>* n,
77                                      int k, bool& rotation_happened) {
78
79               // reads and writes are safe
```

```
80              auto n_data = n->rwRef();
81
82              // calculate node height
83              n_data->height =
84                  max_height(n_data->getChild(0), n_data->getChild(1)) + 1;
85
86
87              // calculate node's balance
88              auto balance = node_balance(n_data);
89
90              rotation_happened = true;
91
92              // for tree modifications (change children etc) use SafeNode
93
94              if (balance > 1 && k < n_data->getChild(0)->key) {
95                  // right rotate
96                  n = right_rotate(n);
97              } else if (balance < -1 && k > n_data->getChild(1)->key) {
98                  //left rotate
99                  n = left_rotate(n);
100             } else if (balance > 1 && k > n_data->getChild(0)->key) {
101                 // left right rotate
102                 n->setChild(0, left_rotate(n->getChild(0)));
103
104                 n_data->height =
105                     max_height(n_data->getChild(0), n_data->getChild(1)) + 1;
106
107                 n = right_rotate(n);
108             } else if (balance < -1 && k < n_data->getChild(1)->key) {
109                 // right Left Rotate
110                 n->setChild(1,right_rotate(n->getChild(1)));
111
112                 n_data->height =
113                     max_height(n_data->getChild(0), n_data->getChild(1)) + 1;
114
115                 n = left_rotate(n);
116             } else {
117                 rotation_happened = false;
118             }
119
120             // calculate new height
```

```
121             n_data->height = max_height(n_data->getChild(0), n_data->getChild
122
123             return n;
124     }
```

4.1.0.3 Assessment of the Concurrent Code

The method is implemented in approximately 107 lines of code (a 60% reduction), with much clearer behavior and many similarities to sequential code. The similarities are so extensive that certain portions require no changes at all to be parallelized through the library. Synchronization code is minimized, and the function is concerned mainly with the logic of the data-structure operation. We regard this as a substantial improvement in programmer productivity.

4.2 Experimental Methodology

Test Machine We measured performance on a machine with two Intel(R) Xeon(R) E5-2697 v3 processors in separate sockets, each with 14 physical cores and support for up to 28 hardware threads through hyperthreading. We restricted our tests to one processor because of the effect of NUMA memory access. The system was equipped with 200 GB of RAM.

We constructed two trees using the library: a simple unbalanced binary search tree and an AVL search tree. For benchmarking, we first create sequentially an instance of each tree with a chosen number of nodes and random keys in a given value range (for example, 10,000 keys in [0, 20000]), with the size of the range twice the number of nodes.

We then create threads pinned to the available processor cores. With given probabilities, they continuously execute tree operations (insertion, lookup, deletion) for five seconds. By keeping insertion and deletion probabilities equal, the tree size does not change significantly from its initial state. We measure completed operations per second in MOPS (millions of operations per second).

The tested implementations are the library's two trees (*avl-framework*, *unbalanced-framework*) and the corresponding manual implementations (*avl-manual*, *unbalanced-manual*) by my thesis advisor, Dimitris Siakavaras.

4.3 Results

Our measurements concern the comparison with a manual RCU-HTM implementation rather than absolute performance, since the library's purpose is to apply that technique automatically. Thus we measure the overhead of using the library.

I, R, and L denote the probabilities of insertion, deletion, and lookup, respectively; *Key Range* denotes the range of node-key values (starting at zero), and *Keys in Tree* is the initial number of nodes in the trees.

4.3.1 Lookup Workloads

4.3.1.1 Small Trees

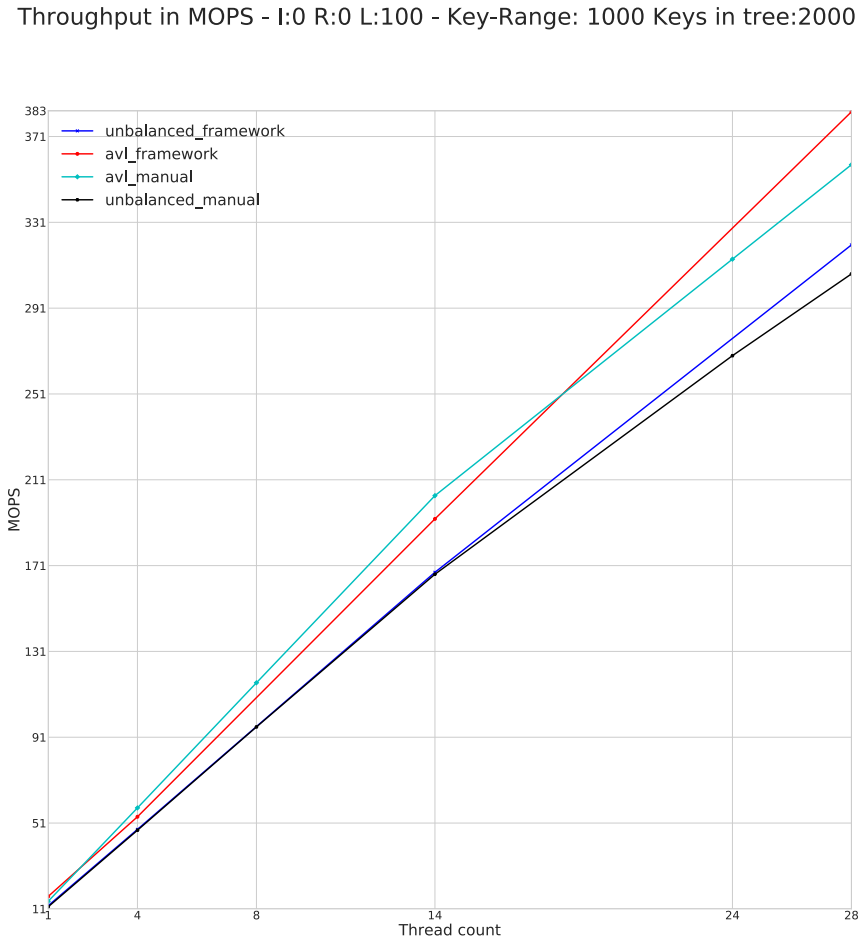


Figure 4.1: 100% Lookups

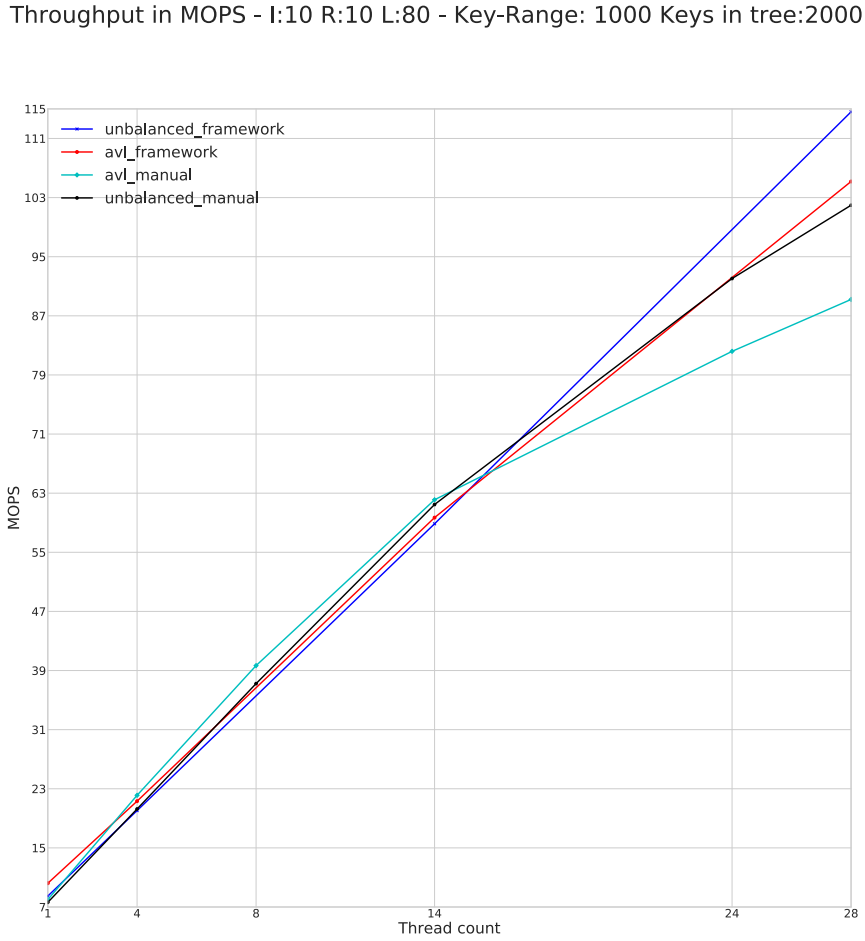


Figure 4.2: 80% Lookups

4.3.1.2 Medium Trees

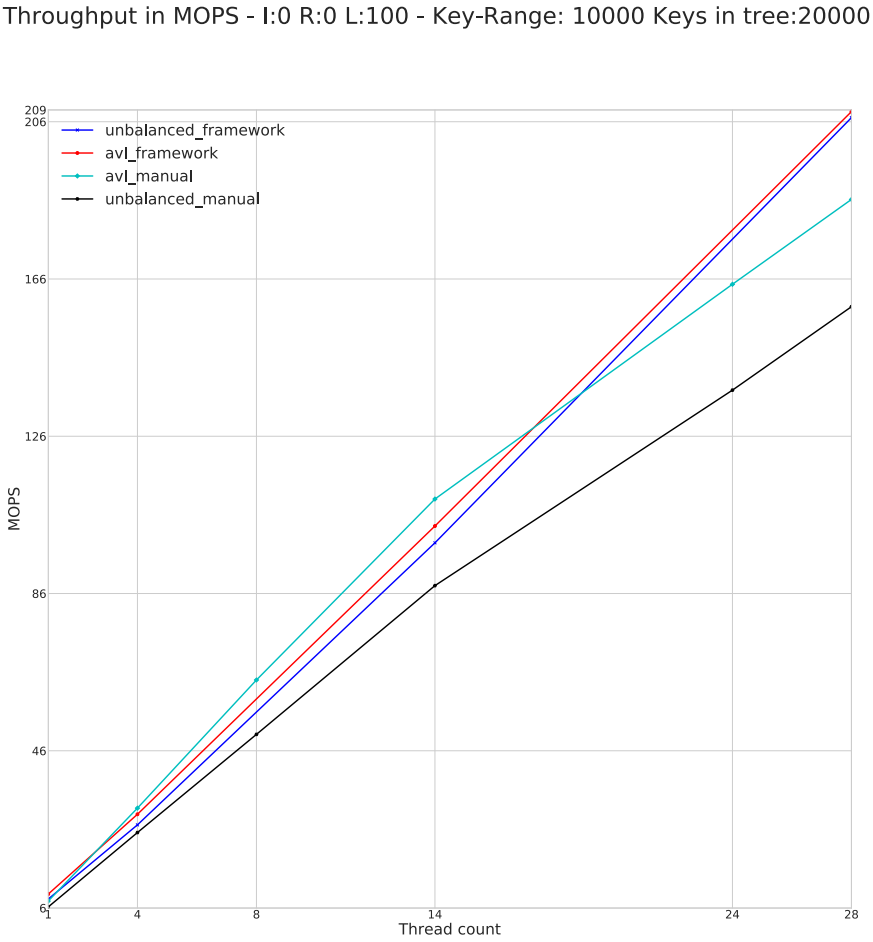


Figure 4.3: 100% Lookups

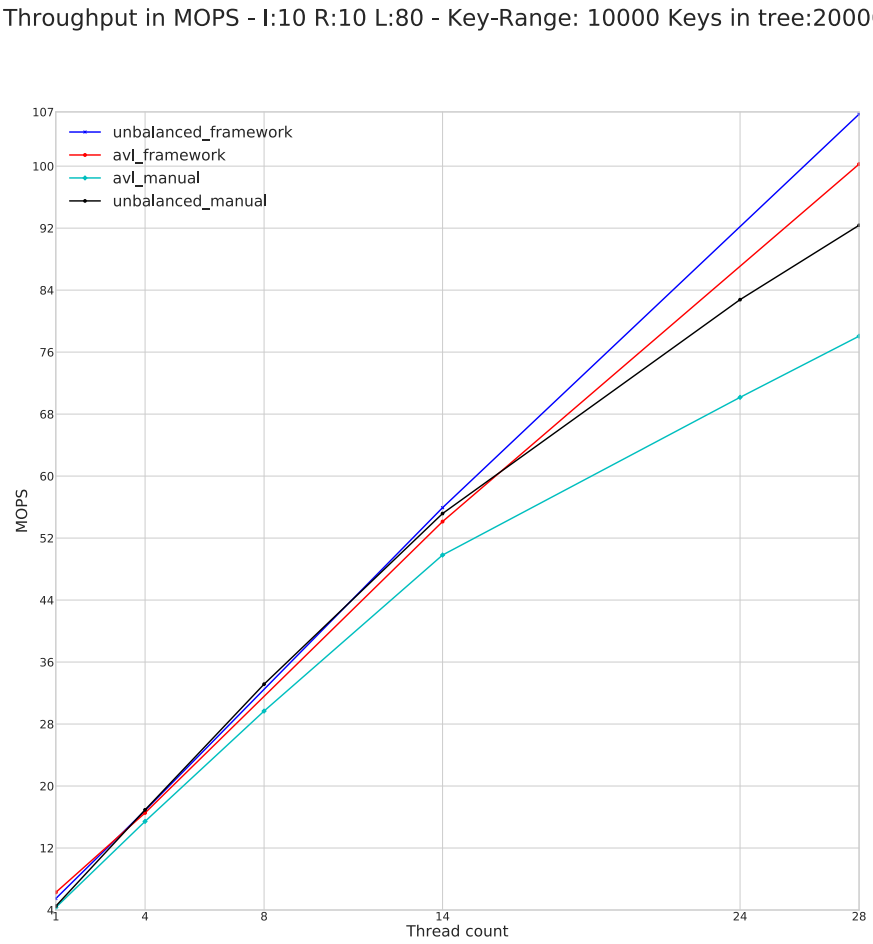


Figure 4.4: 80% Lookups

4.3.1.3 Large Trees

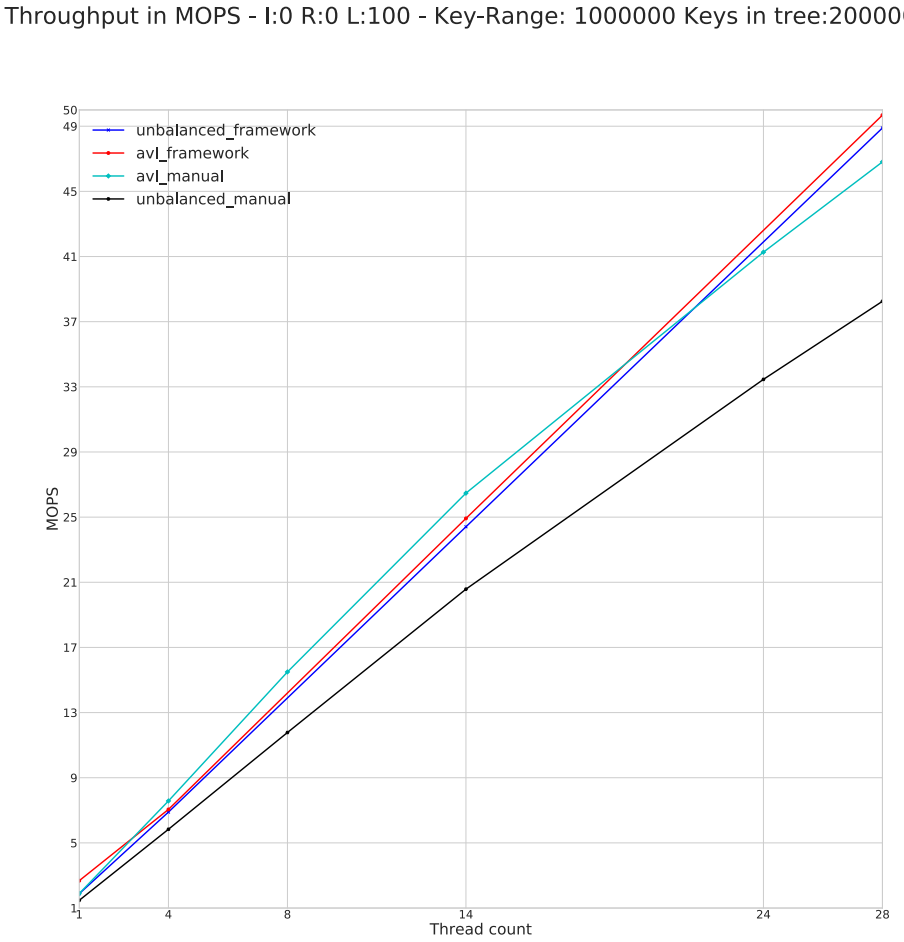


Figure 4.5: 100% Lookups

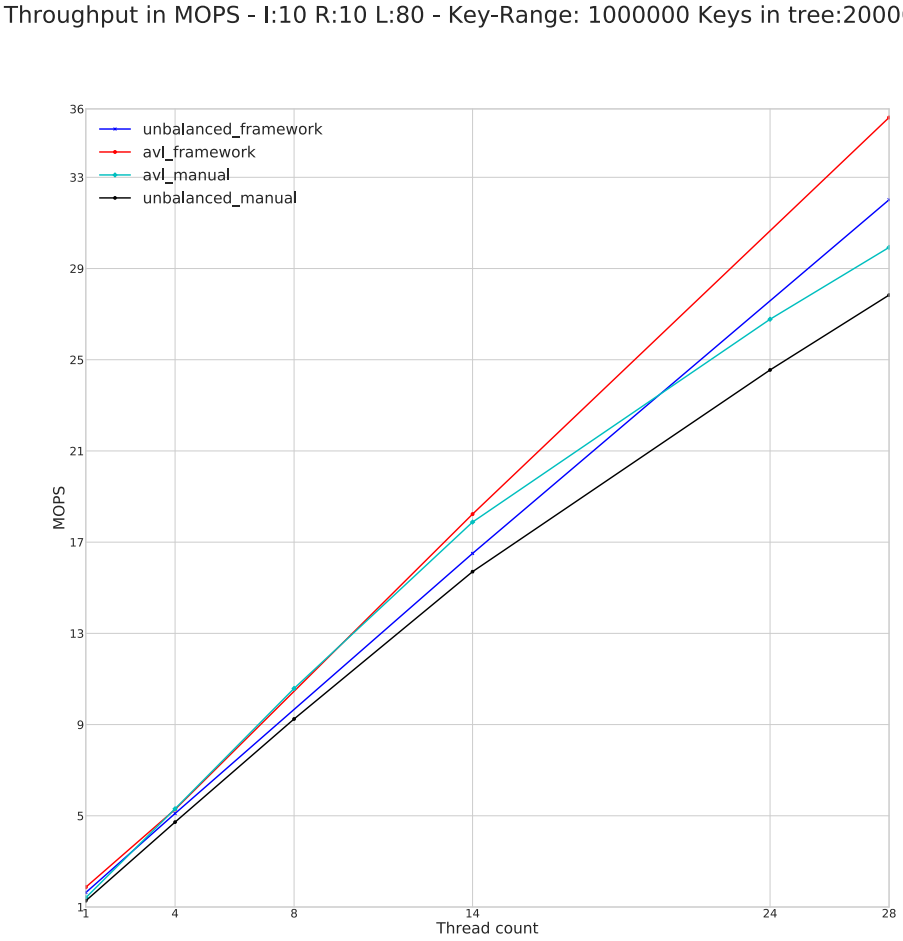


Figure 4.6: 80% Lookups

4.3.2 Balanced Workloads

4.3.2.1 Small Trees

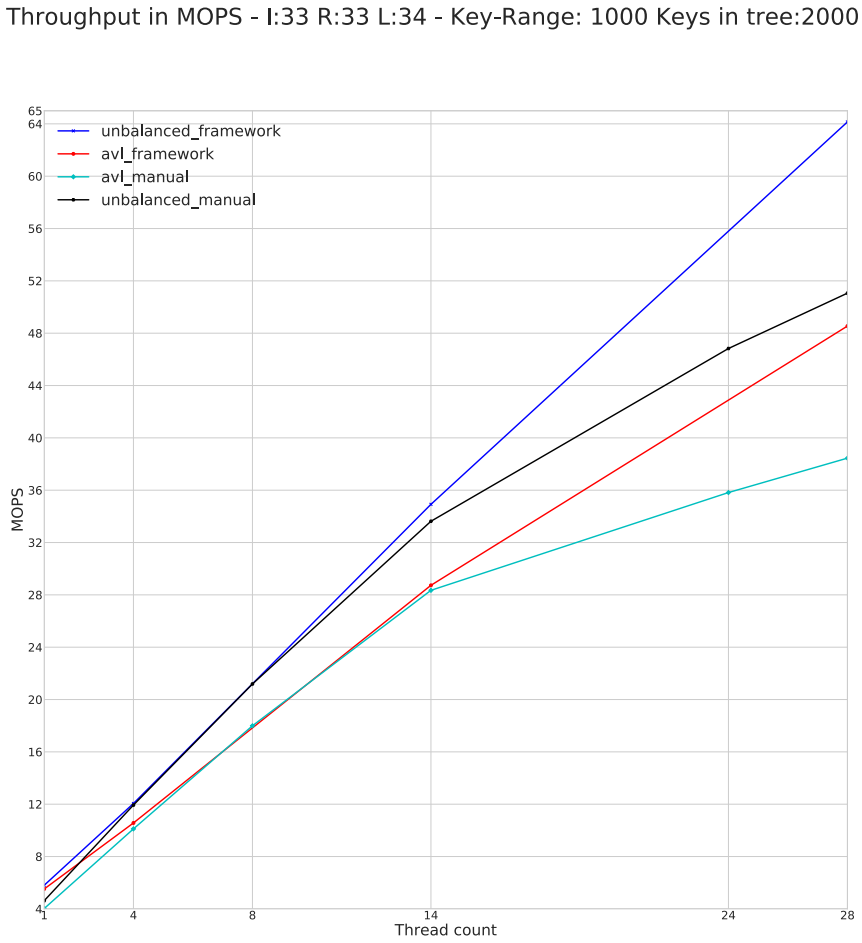


Figure 4.7: Balanced Operations

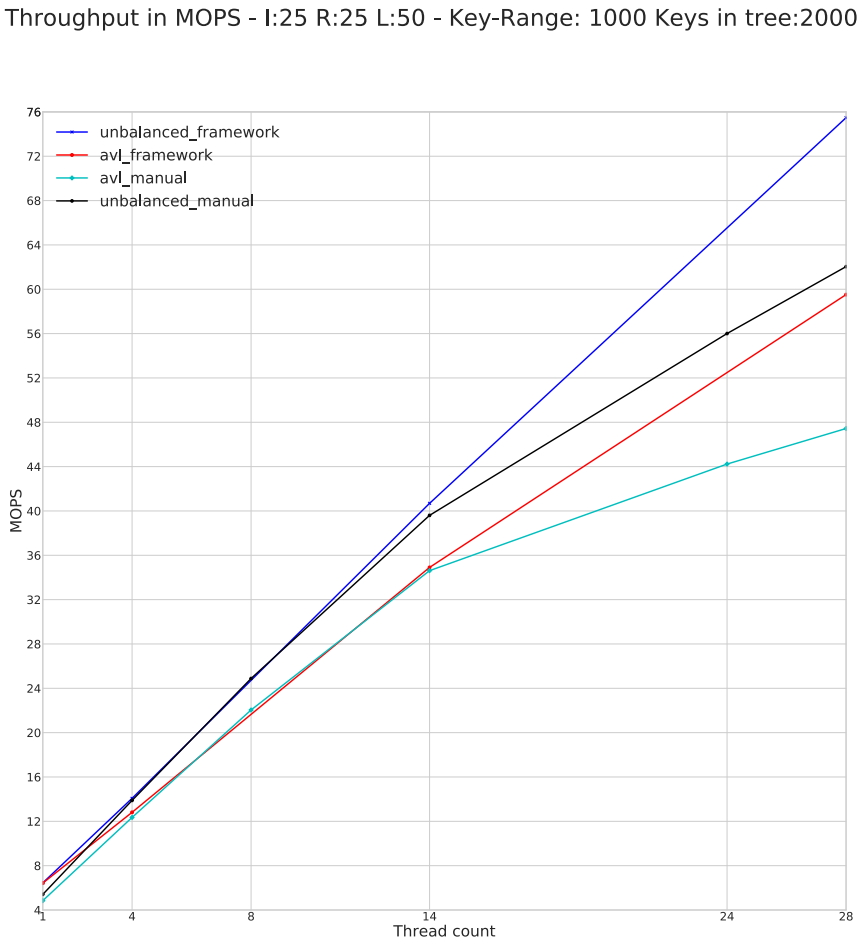


Figure 4.8: 50% Reads

4.3.2.2 Medium Trees

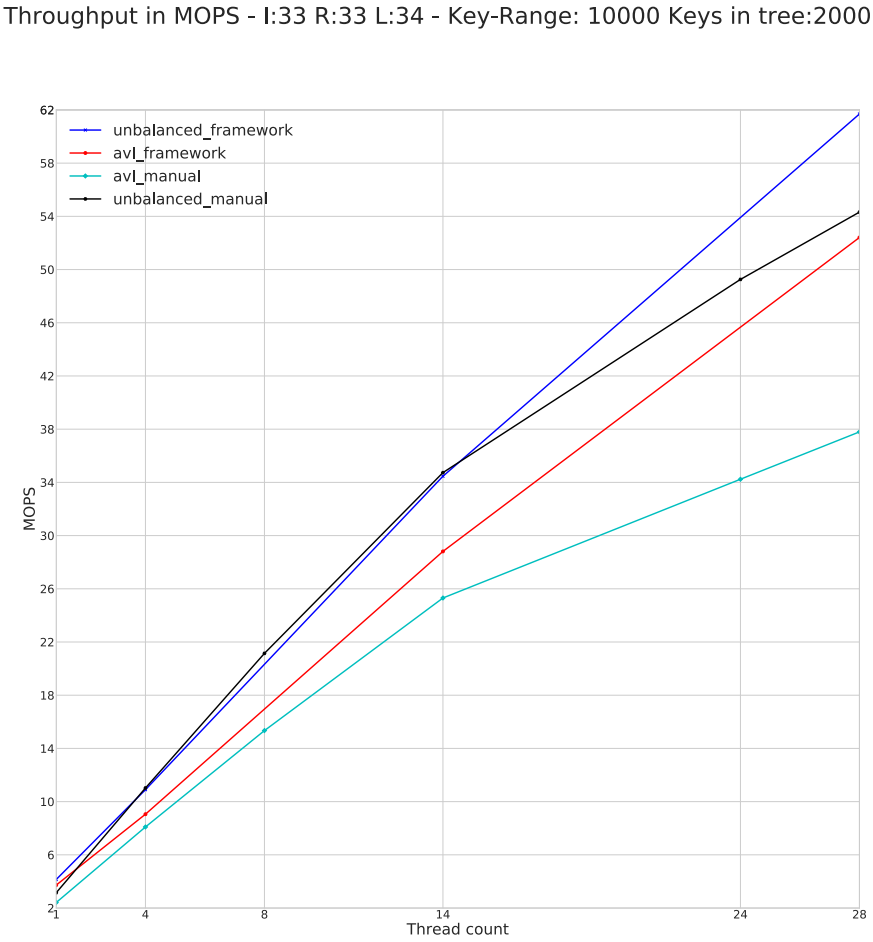


Figure 4.9: Balanced Operations

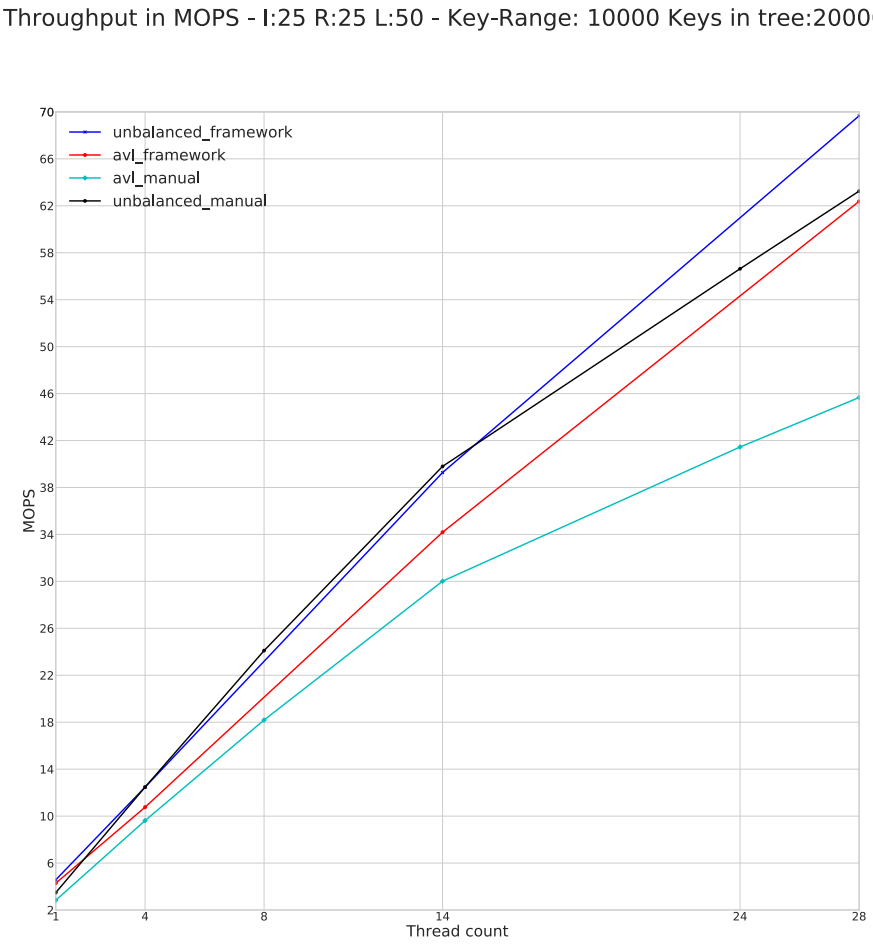


Figure 4.10: 50% Reads

4.3.2.3 Large Trees

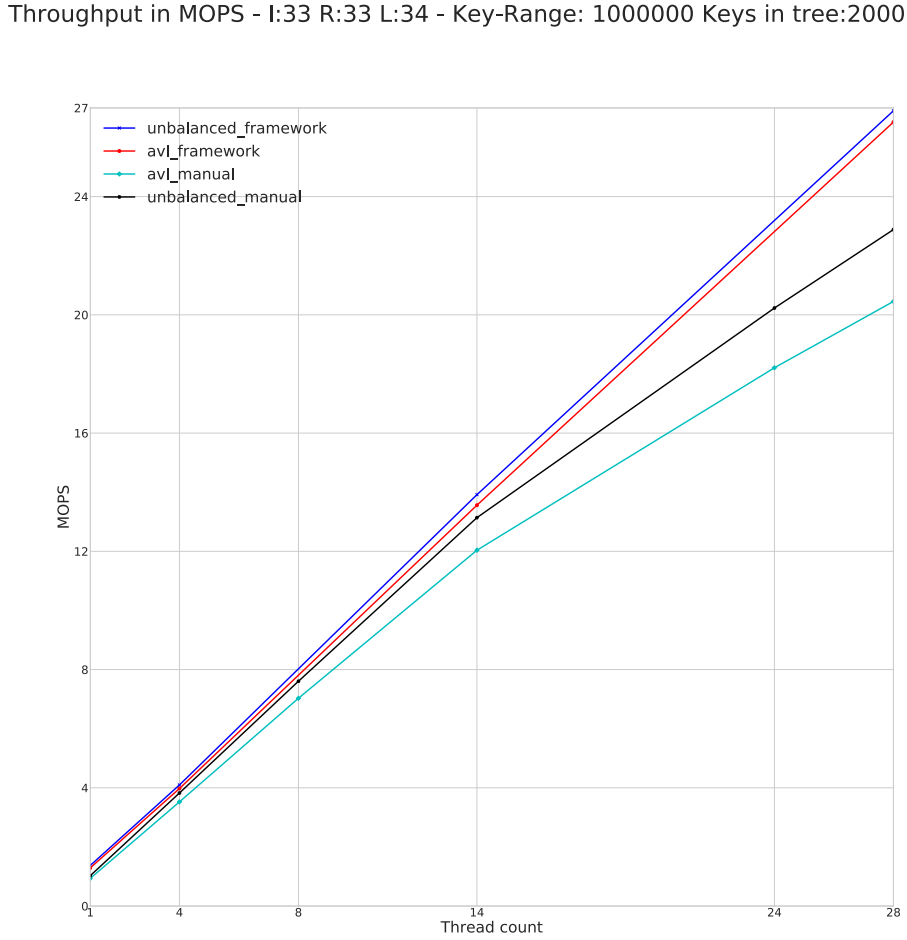


Figure 4.11: Balanced Operations

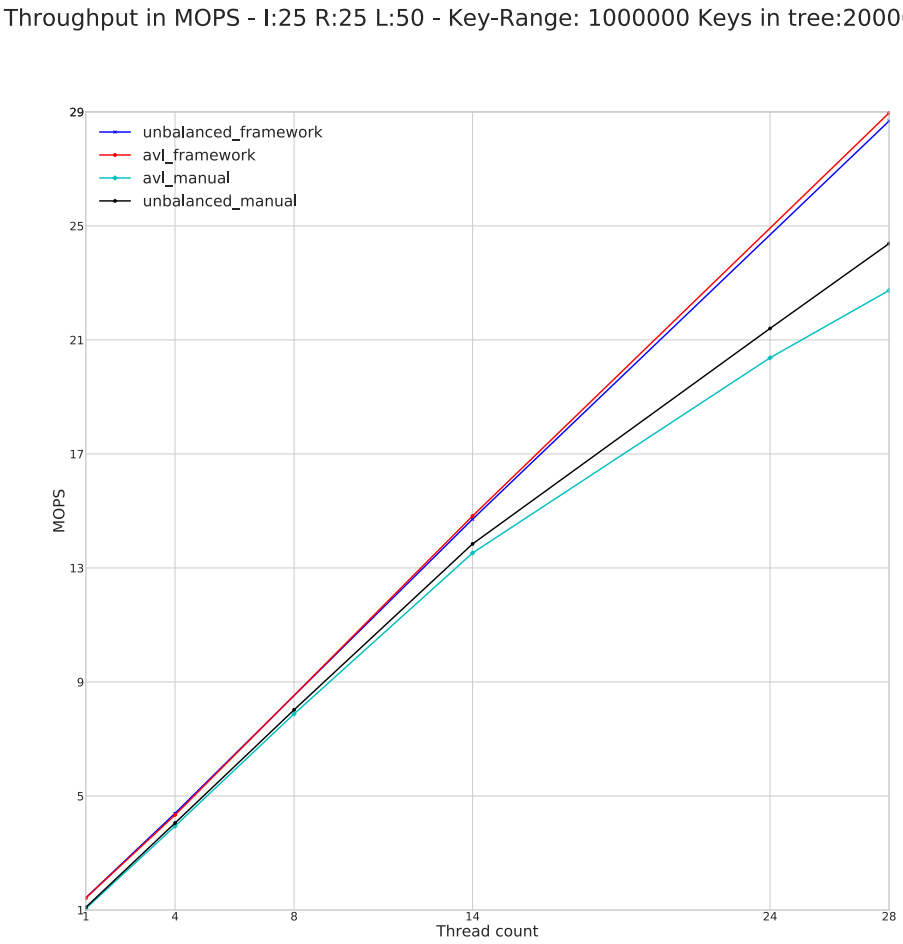


Figure 4.12: 50% Reads

4.3.3 Updates Only

4.3.3.1 Small Trees

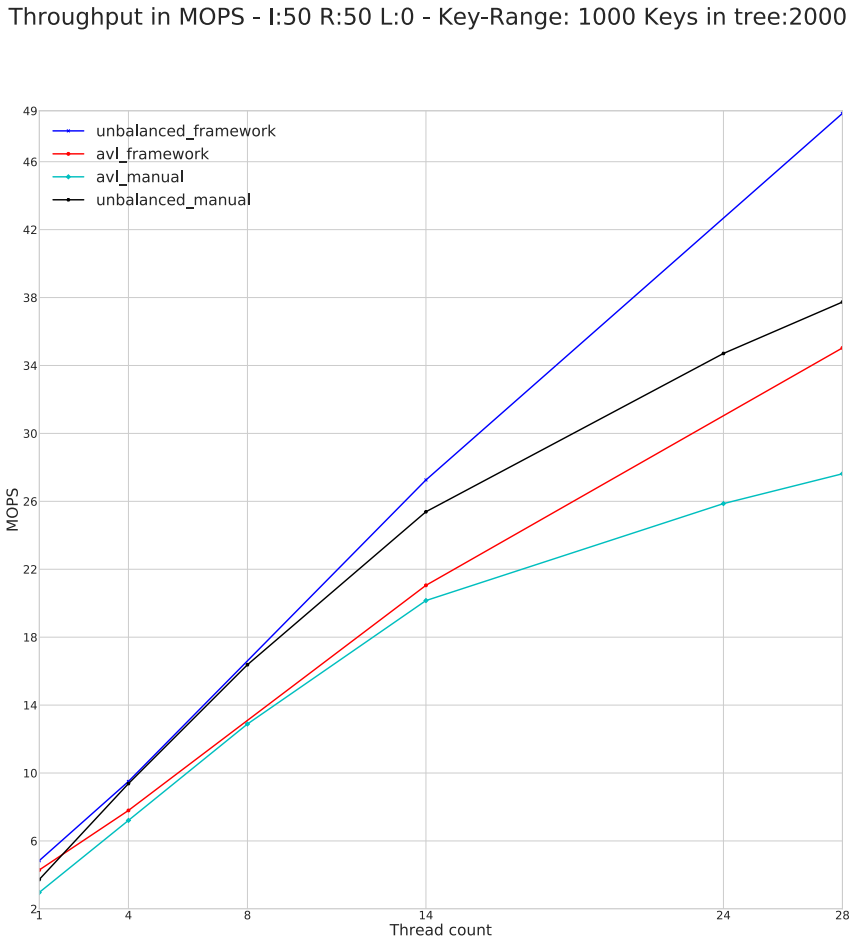


Figure 4.13: Updates Only

4.3.3.2 Medium Trees

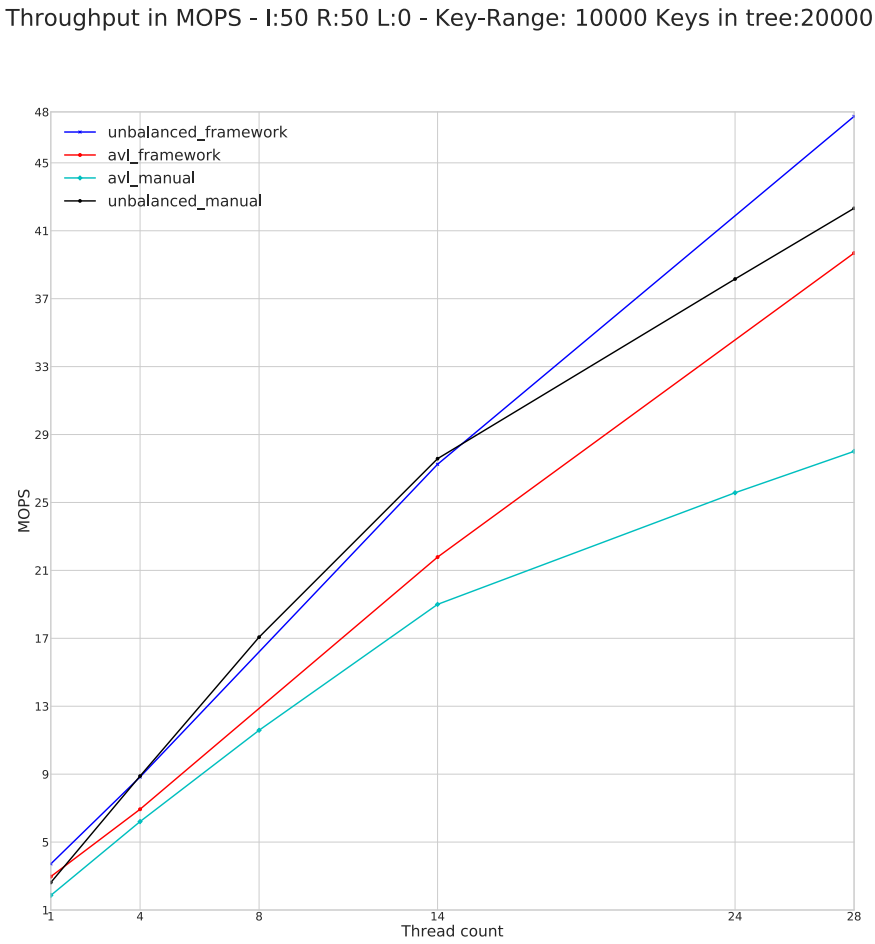


Figure 4.14: Updates Only

4.3.3.3 Large Trees

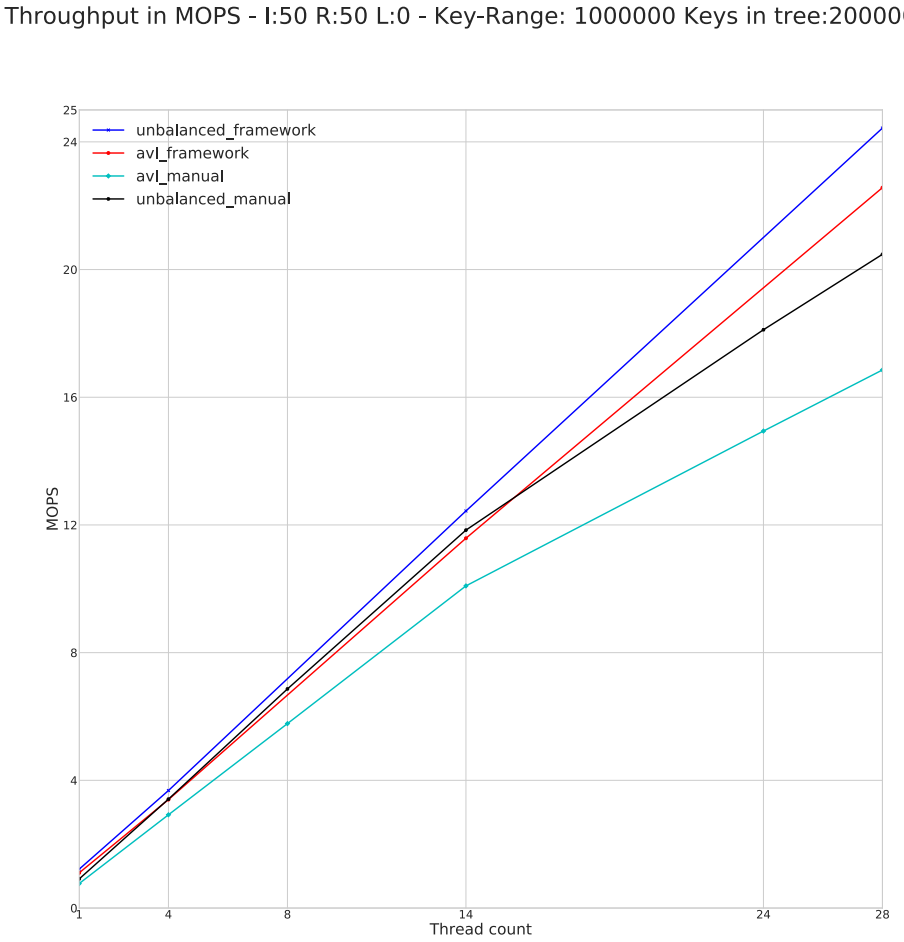


Figure 4.15: Updates Only

4.3.3.4 Performance Analysis

Comparison with manual implementation Across all tree sizes and workload mixes, the **automatically parallelized implementations outperform the manually parallelized implementations**. The precise values do not matter, since the manual implementation is not necessarily ideal (and an ideal one might match the automatic implementation), which is understandable given its complexity. However, these results demonstrate that automation introduces no overhead compared with a manual implementation. Consequently, the resulting structures are as competitive with alternative concurrent implementations as manual RCU-HTM implementations are (see [Sia+17]). Moreover, the library's ease of use helps produce more efficient implementations by hiding the difficulty of optimization.

Overall performance The concurrent structures implemented using the library show nearly linear performance scaling with the number of threads, across all tree sizes and workload mixes.

4.4 Effect of NUMA on Performance

We also ran measurements on a system with two Xeon processors in separate sockets, using a non-uniform memory access (NUMA) architecture. On this system, performance dropped sharply when threads ran on cores belonging to different sockets. As the number of cores used across sockets increased, performance deteriorated toward the level achieved with only one or two threads.

Investigating the issue, we found that NUMA accesses on that system, combined with the need to place the structure in shared memory, caused many page faults. These in turn repeatedly aborted HTM transactions and severely degraded performance.

This issue was not investigated in detail within the scope of the thesis but deserves attention when deploying the library on NUMA systems.

4.5 Factors Affecting the Library's Performance

This section presents several significant factors influencing the performance of structures parallelized using the library. These observations were made while implementing the two evaluated structures.

1. **Alignment of original nodes**. Node alignment plays an important role for small trees (fewer than 100 nodes). Nodes in different subtrees may fall within the same cache line, causing false sharing and transaction aborts and destroying performance at high thread counts. Increasing node alignment reduces this effect, but larger node objects also increase the cost of copying. Experimentation is recommended if performance problems arise.

2. **Number of `getChild` calls**. Each invocation of `getChild` can create a copy when accessing a node not yet placed inside a safe node. Such accesses should therefore be minimized.

3. **Dynamic memory management**. Dynamic memory allocation harms performance because it often triggers transactional aborts, either through page faults or through access to shared internal structures of the memory allocator.

Chapter 5

Conclusions and Future Extensions

5.1 Conclusions

Revisiting the objectives established in the chapter introducing the library, we conclude that the result achieves each of them:

1. The risks and difficulties associated with concurrent code are concealed and handled by the library rather than its user, who writes code resembling a sequential implementation.

2. The final code differs minimally from the sequential version and is easy to understand, naturally focusing attention on the data structure's logic rather than synchronization.

3. The library's performance matches that of an excellent manual implementation of the RCU-HTM technique, making it a zero-cost abstraction.

4. Debugging proves easier because users provide only sequential methods, which they need to check for correctness, provided they comply with the library's conventions.

Furthermore, the library extends the RCU-HTM technique by enabling straightforward parallelization of most tree data structures in practical and academic use. We therefore conclude that the library is an innovative tool capable of greatly facilitating the construction of concurrent tree data structures, improving every stage from design through verification of their consistency.

5.2 Future Extensions

Given the library's extensive capabilities, it is worth highlighting its shortcomings and the future additions that could substantially improve it.

1. The library's main limitation is the absence of a dynamic memory reclamation mechanism. Dynamic memory management in concurrent data structures is a major challenge, perhaps comparable to building this library itself. The library provides mechanisms for allocating safe nodes and user nodes through pools, but it offers no safe means of reclaiming their memory. This is the principal priority for future work.

2. Support for alternative HTM mechanisms beyond Intel's implementation (for example, ARM or a software fallback), since the current implementation is limited to Intel processors that support the TSX-NI instruction set.

3. Porting the library to programming languages other than C++.

Appendix A

Configuration

Compilation Flags

The library provides a number of settings and operating modes controlled through compilation flags.

Numeric Parameters

1. `RCU_HTM_MAX_THREADS`: maximum number of threads (default: 100).
2. `PATH_MAX_LEN`: maximum length of a path from the root to any node (default: 10000).

Operating Modes

1. `TREE_TYPE`: tree type; accepts `SEARCH_TREE` or `GENERAL_TREE` to select the search-tree or general-tree method, respectively.
2. `TM_EARLY_ABORT_POP`: enables early abortion of an operation if validation fails while moving the connection point.
3. `TM_EARLY_ABORT_ALL`: enables early abortion of an operation if out-of-transaction validation fails during execution (also enables `TM_EARLY_ABORT_POP`).

Optimizations

1. Enable safe-node pools by defining `TSX_MEM_POOL`.

2. Enable user-node pools by defining `USER_MEM_POOL`.

3. Use a stack-allocated validation set by defining `PREALLOC_VALIDATION_SET`.

Bibliography

[Bro+10] Nathan G. Bronson et al. "A Practical Concurrent Binary Search Tree." In: *SIGPLAN Not.* 45.5 (Jan. 2010), pp. 257–268. ISSN: 0362-1340. DOI: 10.1145/1837853.1693488. URL: https://doi.org/10.1145/1837853.1693488.

[Des+12] Mathieu Desnoyers et al. "User-Level Implementations of Read-Copy Update." In: *IEEE Transactions on Parallel and Distributed Systems* 23 (2012), pp. 375–382. ISSN: 1045-9219. DOI: http://doi.ieeecomputersociety.org/10.1109/TPDS.2011.159.

[CGR13] Tyler Crain, Vincent Gramoli, and Michel Raynal. "A Contention-Friendly Binary Search Tree." In: *Proceedings of the 19th International Conference on Parallel Processing*. Euro-Par'13. Aachen, Germany: Springer-Verlag, 2013, pp. 229–240. ISBN: 9783642400469. DOI: 10.1007/978-3-642-40047-6_25. URL: https://doi.org/10.1007/978-3-642-40047-6_25.

[CNT14] Bapi Chatterjee, Nhan Nguyen, and Philippas Tsigas. *Efficient Lock-free Binary Search Trees*. 2014. arXiv: 1404.3272 [cs.DC].

[RM15] Arunmoezhi Ramachandran and Neeraj Mittal. "A Fast Lock-Free Internal Binary Search Tree." In: *Proceedings of the 2015 International Conference on Distributed Computing and Networking*. ICDCN '15. Goa, India: Association for Computing Machinery, 2015. ISBN: 9781450329286. DOI: 10.1145/2684464.2684472. URL: https://doi.org/10.1145/2684464.2684472.

[Sia+15] Dimitrios Siakavaras et al. "Performance Analysis of Concurrent Red-Black Trees on HTM Platforms." In: Jan. 2015.

[Nam16] Dmitry Namiot. "On lock-free programming patterns." In: *WSEAS Transactions on Computers* 15 (Jan. 2016), pp. 117–124.

[BER17] Trevor Brown, Faith Ellen, and Eric Ruppert. *A General Technique for Non-blocking Trees*. 2017. arXiv: 1712.06687 [cs.DC].

[Sia+17] D. Siakavaras et al. "RCU-HTM: Combining RCU with HTM to Implement Highly Efficient Concurrent Binary Search Trees." In: *2017 26th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. Sept. 2017, pp. 1–13. DOI: 10.1109/PACT.2017.17.

[WSJ18] Kjell Winblad, Konstantinos Sagonas, and Bengt Jonsson. "Lock-Free Contention Adapting Search Trees." In: *Proceedings of the 30th on Symposium on Parallelism in Algorithms and Architectures*. SPAA '18. Vienna, Austria: Association for Computing Machinery, 2018, pp. 121–132. ISBN: 9781450357999. DOI: 10.1145/3210377.3210413. URL: https://doi.org/10.1145/3210377.3210413.

© 2020 National Technical University of Athens. All rights reserved.